

Toward Physics-Faithful Video Generation

A depth-guided generator, a human-annotated flaw benchmark, and an auditable multi-tool critic

Adobe Research

July 2026

REPORT STATUS AND SCOPE

Scope. This report describes three components built toward physics-faithful video generation: (1) a generator that renders a physics simulation into photorealistic video through a depth/structure control channel; (2) a human-annotated benchmark of prompt-versus-video flaws; and (3) an auditable multi-tool critic that detects such flaws. The three are designed to compose into a loop in which the critic’s judgment refines the generator; **that closed loop is future work and is not evaluated here** (section 5).

What is and is not reported. We report the design of each component and the measured *descriptive* statistics of the artifacts (clip counts, flaw counts, generation configuration, model identifiers). We do *not* report critic accuracy, flaw coverage, false-alarm rates, a comparison between critic variants, or a learned video-quality score for the generator; those are performance claims deferred to a dedicated evaluation. Generator validation in this report is qualitative (visual inspection and adherence to the control signal), not a quantitative quality metric. Section 6 states the measurement protocol these results will follow.

Status vocabulary. **BUILT** present and exercised in the codebase; **PARTIAL** present with limited semantics; **STUB** present but inactive; **NOT INTEGRATED** built but not wired into the full loop; **PROPOSED** designed, not implemented.

Abstract

Text-to-video generators produce clips of high visual quality that nonetheless violate physical common sense—objects interpenetrate, motion ignores gravity, quantities fail to conserve—and often simply fail to depict what the prompt requested. Making generated video *physically faithful* requires three things at once: a generator that can be steered by physical structure rather than appearance alone; a way to measure, reliably and auditably, when a clip is wrong; and ground-truth data about the flaws real generators make. This report presents one component for each.

First, a **physics-guided generator**: a rigid-body simulation is rendered into a per-frame depth (or silhouette) control video and passed to the native control branch of a Wan 2.2-VACE video-diffusion backbone, so that scene geometry and motion are driven by the simulation while the text prompt supplies appearance. Second, a **human-annotated flaw benchmark**: 1,500 AI-generated clips carry human misalignment annotations—each with a severity, a time span, a box track over the flagged interval, the violated prompt fragment, and a free-text rationale—from which we derive a frozen, physics-focused evaluation set. Third, an **auditable multi-tool critic**: a served vision-language reasoner decomposes a prompt into checkable claims, routes each to a frozen open perception specialist that returns localized evidence only, and combines

the outcomes with deterministic rules into a three-valued verdict (*plausible*, *implausible*, *abstain*). The evaluation target is human labels, and no closed or proprietary model produces the critic’s verdict.

The intended end state is a loop in which the critic supplies the reward that refines the generator. We describe each component and its honest implementation status, and we specify the human-label evaluation protocol under which the critic and the loop will be measured; comparative performance numbers are deferred to that evaluation.

1 Introduction

1.1 Physical faithfulness as the goal

State-of-the-art text-to-video models render strikingly realistic clips that still break physical common sense and often omit or distort what the prompt asked for. Physics-IQ quantifies the first problem directly: strong visual realism coexists with a large gap in physical understanding on its evaluated models and tasks [38]. Closing that gap is our long-term objective, and it decomposes into three coupled needs.

- **A steerable generator.** A model that follows *physical structure*—the geometry and motion of a scene—rather than reproducing the surface statistics of its training data.
- **A reliable, auditable detector of flaws.** A measurement instrument that says when a clip is wrong *and* shows why, at the level of individual evidence—because that judgment is the reward signal any improvement loop consumes.
- **Ground truth about real flaws.** A benchmark of the mistakes real generators make, annotated by people, against which any detector is measured.

1.2 Three contributions and the loop they form

This report contributes one component for each need and states how they are intended to compose.

1. **A physics-guided video generator** (section 2) that converts a rigid-body simulation into a depth or silhouette control video and drives a Wan 2.2-VACE diffusion backbone’s native control branch, separating physical structure (from the simulation) from appearance (from the prompt).
2. **A human-annotated flaw benchmark** (section 3): a 1,500-clip corpus of AI-generated video with localized human misalignment labels, and a frozen physics-focused subset used as the evaluation target.
3. **An auditable multi-tool critic** (section 4) that decomposes a prompt into claims, routes each to an open perception specialist returning evidence only, and combines the evidence deterministically into a three-valued verdict.

The end state (section 5) is a closed loop: the benchmark trains and evaluates the critic, and the critic’s verdict becomes the reward that refines the generator. *That loop is future work*; this report builds and characterizes its three parts.

1.3 Non-claims

We do not claim the closed generator–critic loop exists or that the critic has yet improved any generated video. We do not report critic accuracy, flaw coverage, false-alarm rate, or a comparison between critic variants; those await the evaluation of section 6. We do not report a learned video-quality score for the generator—its validation here is qualitative. And we claim novelty for none of the individual building blocks (structure-conditioned generation, prompt decomposition, specialist routing, three-valued abstention); the contribution is their integration toward physical faithfulness.

2 Physics-Guided Video Generation

2.1 Method summary

The generator answers a specific question: can a video-diffusion model be made to follow the *geometry and motion* of a known physical scene, while a text prompt supplies only appearance? The method is three steps. A rigid-body simulation is run once and, at each output frame, the simulator’s own depth buffer and object-segmentation buffer are read, giving two pixel-aligned control videos: a grayscale depth field in which near surfaces are bright, and a binary silhouette of the moving bodies. One of those videos is handed to the video backbone through its native control branch, together with an all-white mask that declares the whole frame to be generated under the control’s guidance, and a text prompt. Nothing about appearance is taken from the simulation: the prompt alone decides material, colour, lighting and setting, while the control video decides where surfaces are and how they move (fig. 1). Validation of this pillar is qualitative throughout (section 2.6).

2.2 Backbone and control mechanism

The backbone is Wan 2.2-VACE-Fun-14B, a two-expert mixture-of-experts video-diffusion model with a native control branch, run through the WanVACEPipeline interface; because the two expert stacks exceed the memory of a single 80 GB accelerator, the experts are CPU-offloaded during generation. Generation takes four inputs: the pre-computed control video, a mask, the text prompt, and one scalar conditioning strength.

The mask decides which parts of the frame are held fixed and which are synthesised. Dark mask regions mark content to be preserved as given; bright regions mark content to be generated under the guidance of the control. Inside the backbone, the mask splits the control video into a preserved part and a generated part, which are encoded separately and concatenated into the control latent stream. We pass an *all-white* mask, so the preserved part is empty and every pixel of every frame is synthesised: the depth field is used purely as guidance and no region of it is copied through into the result. This is the mechanism that lets a grey depth render drive a photorealistic clip without tinting it grey.

The conditioning strength is the scalar by which the control latent stream is added into the denoising stream at each of the backbone’s control layers. One value applies uniformly to every control layer; a per-layer schedule is also accepted by the interface but is not used here. Turning it down weakens how tightly the generated motion is bound to the simulated motion without changing anything else about the request.

Bouncing ball

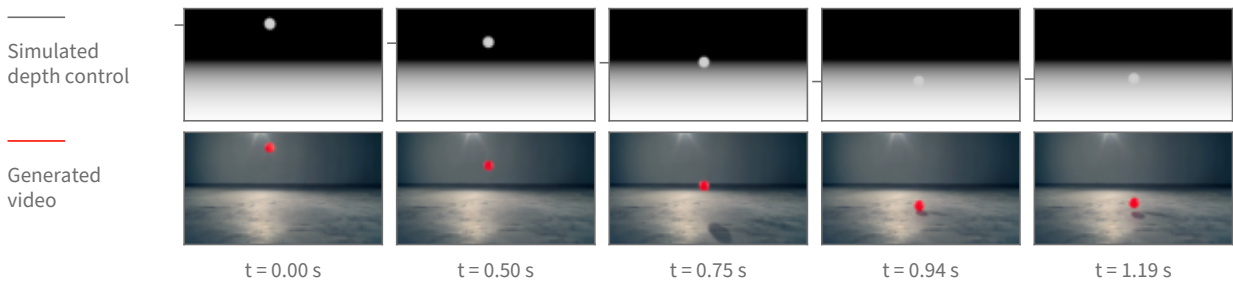


Figure 1: Physics-guided generation, shown on real frames of one bouncing-ball clip. **Top:** the depth control video rendered from the rigid-body simulation, the only geometric input the backbone receives. **Bottom:** the clip generated from it, where the text prompt supplies appearance alone. Both rows are sampled at the same five timestamps of the same 5.06 s, 16 fps, 848×480 clip; the timestamps are chosen by tracking the ball and taking the turning points of its vertical path, so the columns show the drop, the floor contact and the rebound rather than five evenly spaced instants. Tracking the ball independently in each row, the generated path follows the control’s with a correlation of 1.0000 horizontally and 0.9997 vertically over all 81 frames, and a median positional disagreement of 0.7 pixels across the frame and 4.7 pixels down. The tick beside each control frame marks the ball’s measured centre height; the frames themselves are unaltered.

Appearance comes only from text. The prompt names the target scene in the ordinary way (“a bright red rubber ball bouncing on a polished concrete studio floor”), and a single fixed negative prompt—shared by every clip—pushes away from the rendered look the control video literally has, listing cartoon, CGI, render, grayscale and flat shading among the things to avoid. No appearance reference image is supplied. Sampling is deterministic given the seed, and the seed is recorded per clip.

Two properties of this arrangement are worth stating plainly. First, the backbone consumes a control video that was *computed*, not *estimated*: no monocular depth predictor sits in the loop, so the geometry the generator follows is exactly the simulator’s geometry, with no estimation error of its own. Second, this is a control-branch mechanism rather than image-to-image editing: it is neither an SDEdit-style noised re-synthesis [35] of a rendered source nor a copy of source pixels, which is what separates it from conditioning schemes that carry a synthetic source’s look into the output.

2.3 Rendering the control signal

One simulation, several registered views. A scene is described as data—a list of moving bodies with initial positions and velocities, a list of static surfaces, and a camera placed by a look-at rule—and the same engine turns any such description into a simulation. Dynamics are integrated in MuJoCo [47] under gravity at 9.81 m/s^2 with a one-millisecond solver step, a fourth-order integrator, and an elliptic friction cone. Output frames are produced on a coarser clock: the requested scene duration is divided into the frame budget, and the solver is advanced by that many sub-steps between frames. At every output frame, and from the *same* solver state, three renderers are queried in turn—a shaded colour view, the depth buffer, and the object-segmentation buffer. Because they read one state, the resulting videos are registered pixel for pixel, so a depth control

and a silhouette control for the same scene describe the same geometry and can be substituted for one another without re-simulating.

From depth buffer to control video. The depth buffer holds distance from the camera. It is turned into a control video by an inverted, clipped affine map: distances are mapped so that near surfaces are bright and far surfaces dark, which is the polarity monocular depth estimators produce and therefore the polarity the control branch was trained to read. The mapping’s endpoints are robust percentiles—the first and the ninety-seventh—of the depth values in the scene, computed after discarding the far-clip constant that the background sky writes into the buffer, so that a single distant constant cannot compress the whole range into a few grey levels.

Why the mapping is fixed for the whole clip. The endpoints are computed once over all frames of the clip, not per frame. This is a substantive choice, not a detail: it makes the assigned grey level a function of distance alone, so the same distance reads as the same grey in every frame, and a change in brightness therefore means a change in distance. If the endpoints were recomputed per frame—which is the convention monocular depth estimators follow, since they emit one relative map per image—the grey assigned to a distance would depend on the depth distribution of that particular frame. A body holding its distance while other content enters or leaves the view would then change brightness, and the control branch has no way to distinguish that from the body actually moving toward or away from the camera. Fixing the mapping across the clip removes that ambiguity by construction. A per-frame variant is still rendered alongside the fixed-mapping one so the two can be compared directly on the same simulation.

Boundary treatment. A simulator’s depth buffer has ideal, hard edges: a silhouette boundary is a one-pixel step. Estimated depth maps, which is what the control branch has seen, do not look like that—their boundaries are soft. Each grayscale frame therefore receives a small symmetric Gaussian blur before encoding. Because the kernel is symmetric, the half-intensity contour of an edge stays where it was, so the smoothing changes how the boundary is presented without displacing the geometry it represents.

Silhouettes, and when they replace depth. The segmentation buffer labels each pixel with the geometry that produced it. Collecting the labels that belong to the simulation’s moving bodies and painting them white on black yields a silhouette control: the moving bodies’ outlines over time, with no distance information at all. Depth is the default because it carries both outline and distance ordering. It stops being informative when a scene is flat—when every object rests on one near-horizontal surface at nearly the same distance from the camera, the depth field is almost constant across the objects and their surroundings, and the control has little geometric contrast to offer. Measured on the rendered controls, the depth separation between a moving body and the surface immediately around it is about twenty grey levels for a ball bouncing on a floor and about a hundred for a ball arcing through open air, but only about fourteen for a rack of balls on a tabletop. For those flat layouts we substitute the silhouette, which restores a maximum-contrast figure-and-ground signal; the price is that distance ordering between the objects is discarded, and the generator has to recover it from the prompt and from ordinary scene priors.

Setting	Value
Backbone	Wan 2.2-VACE-Fun-14B (two-expert MoE), diffusers WanVACEPipeline, bf16
Memory	experts CPU-offloaded (two-expert weights exceed an 80 GB accelerator)
Control signal	MuJoCo z-buffer depth (globally normalized); object silhouette for flat scenes
Conditioning	control video + all-white mask + text prompt; no reference image
Resolution	848 × 480 (flow-shift 3.0)
Frames / rate	81 frames at 16 fps
Sampler / steps	UniPC, 50 steps
Guidance scale	5.0
Conditioning scale	1.0 (swept over {0.5, 0.8, 1.0})

Table 1: Deployed generation configuration, from the generation scripts. A small checkpoint provides a pre-flight sanity gate and is not used for final clips.

A timing property of the renders. The frame budget and playback rate are fixed—eighty-one frames written at sixteen frames per second—while the simulated interval differs per scene. The clip therefore plays the simulated motion back stretched in time, by factors between roughly $1.8\times$ and $2.3\times$ for the scenes used here. The trajectory shapes and the contact events are the simulation’s; their absolute rate in the finished clip is not, and a downstream measurement of speed or duration must be read against the render clock rather than the simulator’s.

Relation to structure conditioning generally. Driving synthesis from an explicit geometric control rather than from text alone follows the lineage of ControlNet for images [57] and motion and structure control for video [25, 49]. What differs here is the provenance of the control: it is not estimated from a photograph or drawn by hand but computed from a physics solver, so the structure the generator is asked to follow is physically consistent by construction.

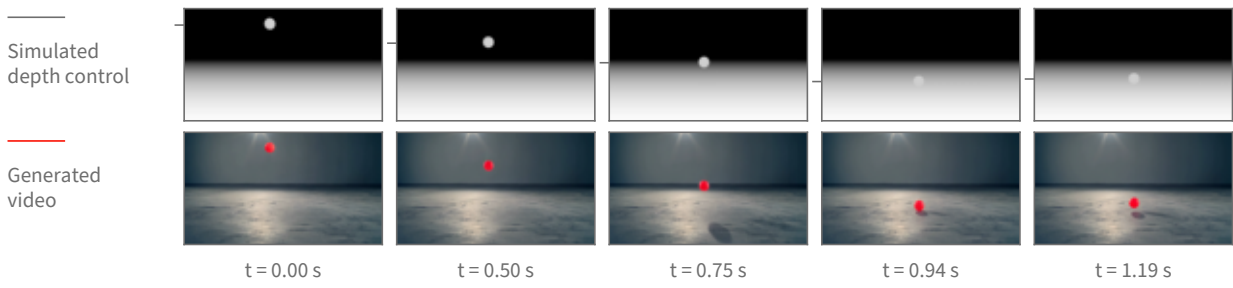
2.4 Pipeline and configuration

Table 1 lists the operating configuration. Generation runs as a staged GPU job: control videos are rendered, a small checkpoint provides a fast sanity gate, the 14B checkpoint generates the clips (data-parallel when more than one GPU is available), and the clips are streamed to storage.

2.5 The scene set is not only balls

The two cases above are single rigid bodies because those are the scenes whose trajectory can be checked against a closed-form path. The scene set itself is much wider: 1,314 scenes across 66 kinds. Before any of them was rendered photorealistically, 652 of the underlying simulations were watched and 591 judged to show the intended physical behaviour clearly enough to be worth generating from; that review is of the simulations, not of the generated video. fig. 3 shows twelve of those kinds as real generated frames—cloth draping and crumpling, a flag in wind, a swinging rope and a hanging chain, grains pouring into a heap and draining through a funnel, a ball dropped

Bouncing ball



Ball thrown across a field

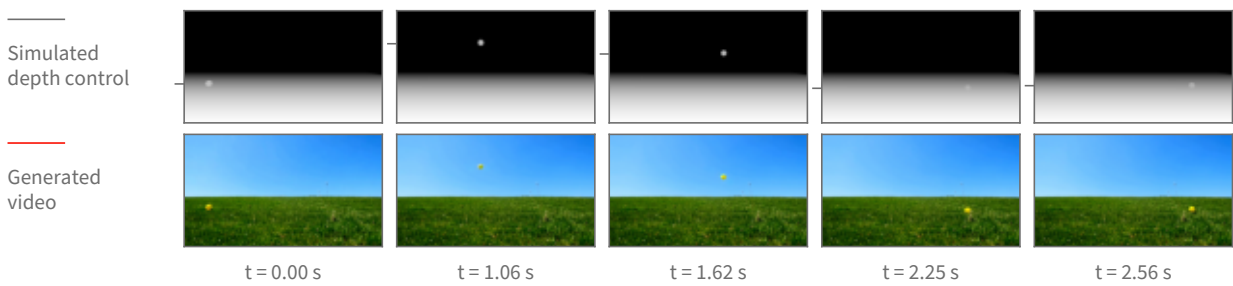


Figure 2: The same mechanism across two scenes with very different requested appearance—an indoor studio and an outdoor field—each shown as its depth control above and the resulting generated clip below. Each scene has its own timestamps, chosen from the turning points of that scene’s tracked vertical path so every column carries motion. The appearance changes completely with the prompt while the motion stays the simulation’s: for the thrown ball the generated and control paths agree to a correlation of 1.0000 horizontally and 0.9998 vertically, with a median disagreement of 0.7 pixels across and 1.1 pixels down, over the 63 frames in which the ball is measurable in both before it leaves the frame.

into a pit of balls, a loaded boat, a cascade down a slope, toppling dominoes and a chaotic two-arm linkage. fig. 4 shows three of them the same way fig. 2 shows a ball: the simulated depth control above, the generated clip below, at matched timestamps.

One limit should be read off these figures explicitly. Everything here comes out of the same rigid-body engine, so the deformables are that engine’s particle-and-link primitives: cloth is a sheet of particles joined by springs, a rope or chain is a short row of linked segments, sand and marbles are many small solid bodies in contact, and floating is a buoyancy force on a solid body in a still fluid volume. None of it is a fluid solve, none of it is smoke, and none of it is a finite-element soft body. The scene captions name the primitive rather than the phenomenon for exactly this reason.

2.6 What it achieves, and how we validated it

Generation was validated *qualitatively*: by visual inspection of the output clips and by a lightweight color-blob trajectory tracker that checks whether the generated object follows the control’s path (fig. 2). There is no learned or quantitative video-quality score (no FVD, no human-preference model, no critic score) applied to this line, and we make no such claim. Under this qualitative bar, single-object scenes—a bouncing ball, a projectile arc—render cleanly with the object following

One frame of the generated video per scene, at the point half the clip's motion has happened.

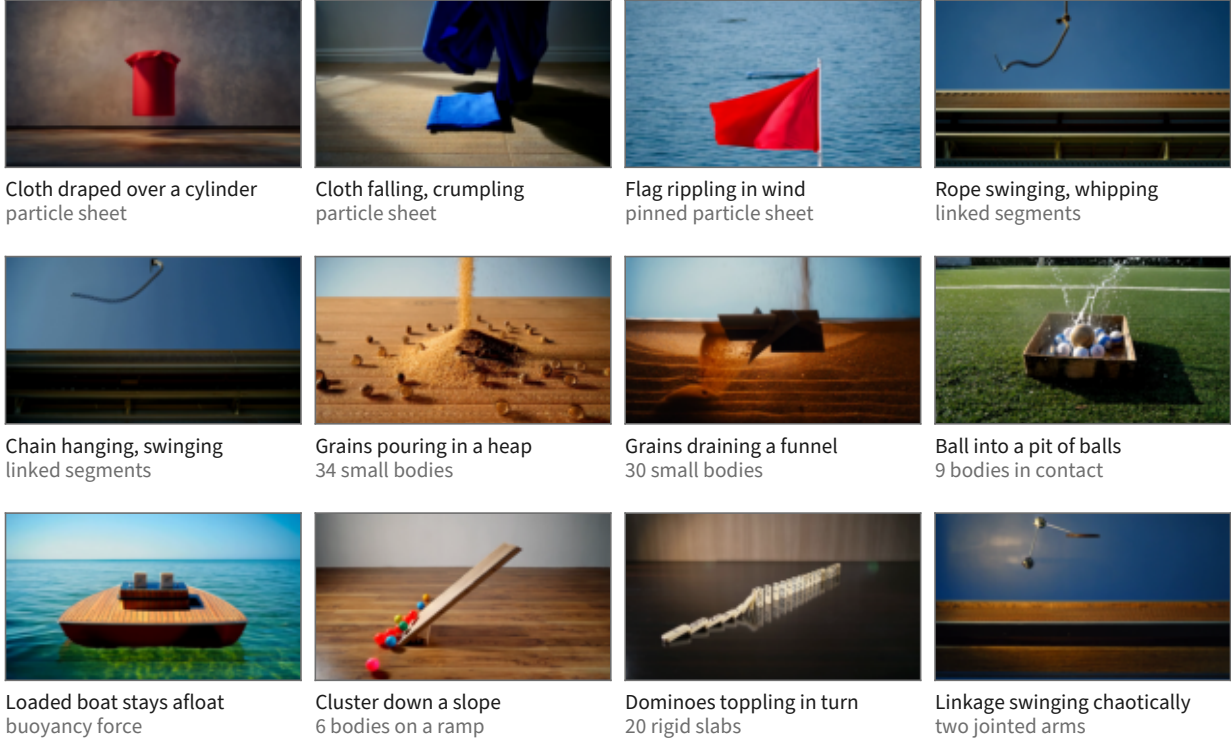


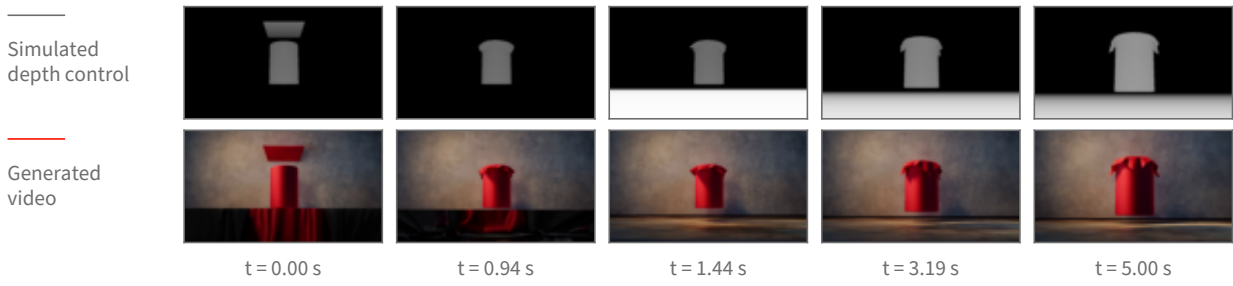
Figure 3: Twelve of the 66 scene kinds, one real generated frame each. The frame shown is computed, not picked: for every clip the frame-to-frame change is accumulated over the whole clip and the tile is the frame by which half of that total change has happened, which lands mid-drape, mid-swing or mid-pour rather than on the first frame. The second caption line names the physics primitive, and every body count in it is the scene’s own parameter. The simulation each clip was generated from passed the pre-generation review described in the text; the generated clips themselves were checked by eye for this figure, which is a weaker guarantee than a systematic review. Two candidate tiles were cut after tracking their control renders showed the phenomenon their label would have claimed does not occur in the clip.

the simulated trajectory. Multi-object interaction (e.g. a billiards break) is *not* solved: neither the deployed backbone nor the evaluated alternates produce a clean multi-object collision, and we report it as an open problem rather than a result.

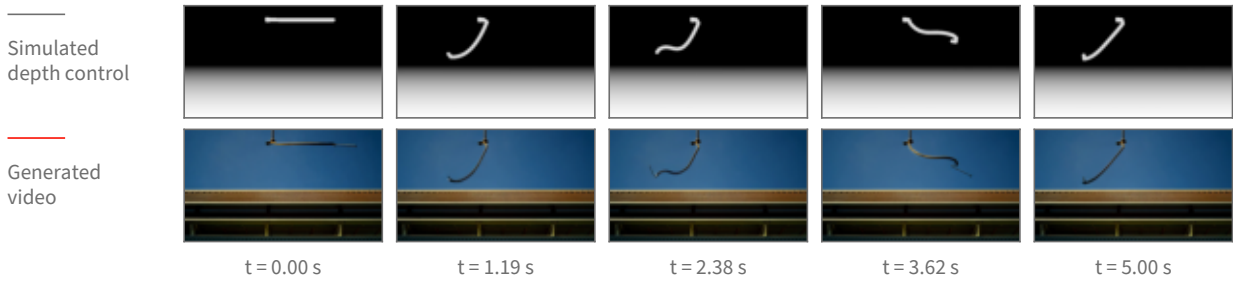
2.7 Design choices

Two choices define the deployed recipe and are stated as design decisions. (1) **Native control over source re-synthesis.** A control branch that consumes a pre-computed depth video keeps the generator from importing a rendered source’s synthetic look. (2) **No reference image.** The pipeline supports an optional appearance reference, but the deployed recipe omits it; including it degraded single-object trajectories in our inspection.

Cloth falling and draping over a cylinder



Rope swinging from a fixed anchor



Grains draining through a funnel

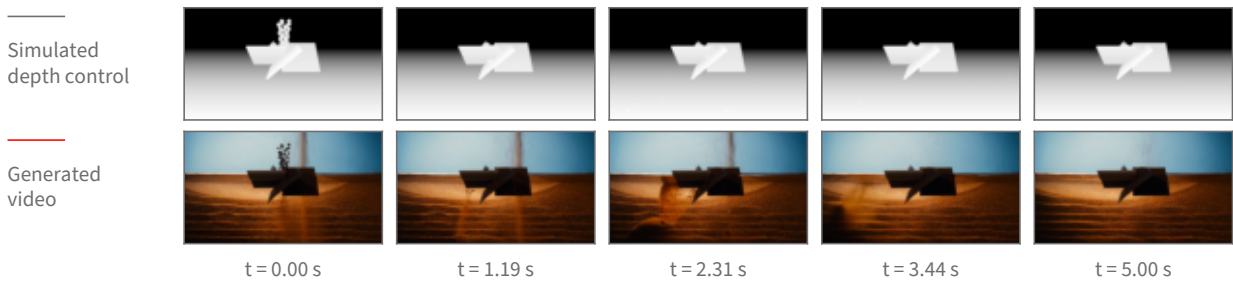


Figure 4: Three deformable and granular scenes shown as fig. 2 shows a ball: the simulated depth control above, the generated clip below, at matched timestamps. Columns are placed at equal steps of the clip’s accumulated change so no column falls in a static stretch. These three cases are the ones whose depth control still shows its object in every column—in a depth picture an object that settles onto the floor takes on the same grey as the floor behind it and disappears from the control, so how much of each control frame is distinguishable from its background is measured, and cases whose control goes blank are not shown as pairs.

2.8 When the control can be released

The control does not have to be applied for the whole of denoising. A diffusion model builds a video over a sequence of denoising steps that starts from almost pure noise and ends at a clean frame, and the control signal is added in at every one of those steps. That raises a question with both a practical and a scientific side: if the control is switched off partway through, does the motion it has already induced survive to the finished clip? Practically, the steps after the release are free of the control, so the model is free to invent appearance there. Scientifically, the earliest release point that still preserves the motion is the point at which the model has *committed* to that motion.

Method. We pick a step, keep the control at full strength for every step before it, and set the control’s contribution to exactly zero from that step onward. Nothing else changes: same simulation, same control video, same prompt, same sampler, same seed. The comparison is always against the same clip generated with the control never released, so the release point is the only difference between the two.

The release point is reported as a noise level, not a step number. A step index is not a property of the model; it is a property of the sampling schedule we happened to choose. The schedule maps each step to a noise level, and that mapping changes when the number of steps or the schedule’s shape changes. Concretely, on the schedule used here, step 8 of 40 sits at a noise level of $\sigma = 0.952$; on a differently-shaped 40-step schedule the same step 8 sits at $\sigma = 0.923$, and the noise level 0.952 has moved to step 5. Reporting “released at step 8” would therefore describe our schedule rather than the model, so every release point below is given as the noise level σ at which the control was switched off, with the step index alongside it only as the label of the run.

What is measured. Both the control video and the generated clip are tracked frame by frame, giving the object’s path in each. *Retention* is how much of the simulated motion the generated clip keeps: the agreement between the generated path and the control’s path, measured separately across the frame (horizontal) and up and down (vertical), because those two directions do not behave the same way. Agreement is a correlation over the frames in which the object is located in both videos, so a value near 1 means the generated object moved the way the simulation did. Two windows are reported, because a single whole-clip number hides where a failure lives: the *airborne* window, up to the frame at which the simulated object first reaches the ground, and the *after-contact* window from that frame to the end. The contact frame is not hand-set per scene; it is read off the control’s own vertical path as the first return to the floor after the object’s highest point. Alongside retention we count two failure modes. A *collapse* is a clip whose horizontal agreement falls below 0.7, meaning the sideways motion no longer resembles the simulation’s. A *phantom* is the specific disagreement in which the whole-clip number reads as broken (below 0.7) while the airborne window alone reads as clean (above 0.9) — the signature of a clip whose flight is right and whose ending is wrong. Both thresholds were fixed before scoring.

Result: the flight survives an early release; the ground contact does not. Figure 5 shows the measurement. Two scenes were repeated over 16 seeds each at one release point, $\sigma = 0.952$, in two variants (with and without a reset of the sampler’s internal history at the release), against the never-released baseline: 96 clips in total, 16 per cell. Figure 5 shows that comparison. Releasing the control at $\sigma = 0.952$ leaves the airborne flight of the thrown ball essentially untouched — per-seed horizontal agreement drops by 0.003 relative to never releasing (95% confidence interval 0.001 to 0.004, $n = 16$ paired seeds), which is a real but negligible difference. Over the whole clip the same release costs 0.13 horizontally (95% CI -0.02 to 0.28 , $n = 16$) and 0.18 vertically (95% CI 0.07 to 0.29 , $n = 16$). The gap between those two windows is the finding: what an early release breaks is not the arc but what happens after the ball reaches the ground. Scoring only the frames after the simulated contact, the generated ball’s horizontal motion runs *opposite* to the simulation’s on 9 of 16 seeds when the control is released at $\sigma = 0.952$, and on 0 of 16 when it is never released — the simulation rolls the ball forward and the generator, cut loose, rolls it back. The whole-clip collapse count follows: 3 of 16 seeds at $\sigma = 0.952$ versus 1 of 16 never released, and the phantom

At one release point (noise level 0.95), 16 seeds per bar

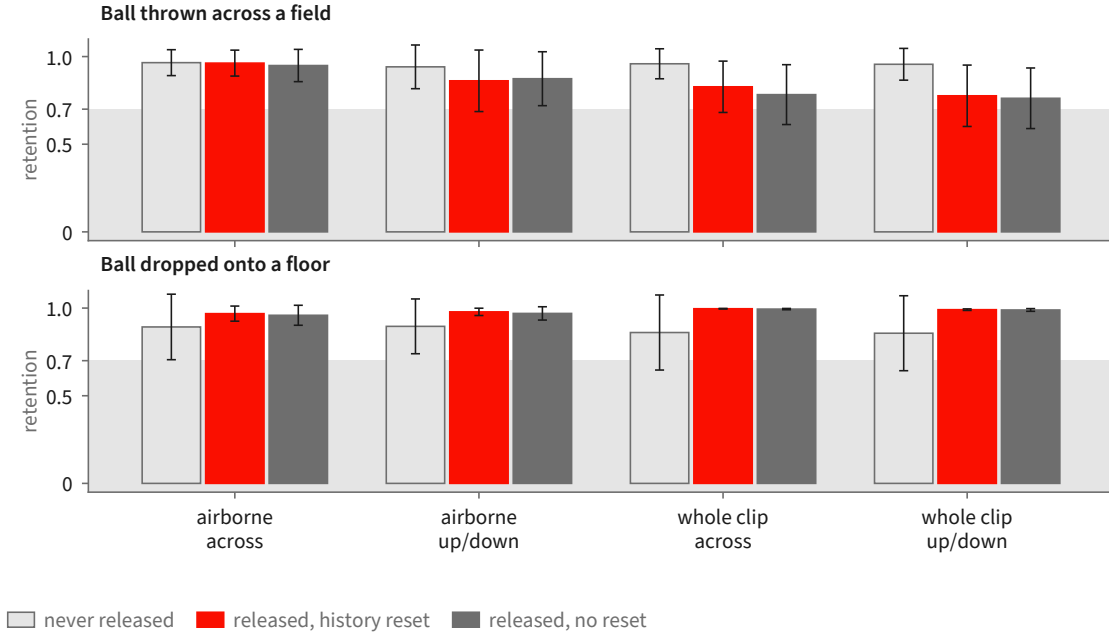


Figure 5: The seeded measurement at the single release point $\sigma = 0.952$, 16 seeds per bar, error bars showing 95% confidence intervals of the mean. Retention is the agreement between the generated object’s tracked path and the simulation’s, so 1.0 is “moved the way the simulation did”. The three bars in each group are the never-released baseline and the two release variants, which differ only in whether the sampler’s internal history is also reset at the release. The airborne window is indistinguishable from never releasing; the whole-clip number is not, and the direction that suffers is the horizontal one. Grey band marks the region below the pre-registered 0.7 break line.

count is 2 of 16 versus 0 of 16, so the collapses that do appear are predominantly ending failures on clips whose flight was clean.

Result: a later release is clean, and the vertical direction is the robust one. Sweeping the release point across six settings on a single seed for three scenes gives the shape of the curve (fig. 6). For the thrown ball, releasing at $\sigma = 0.952$ gives an after-contact horizontal agreement of -0.63 (the ball travels the wrong way) with the airborne window still at 1.00; releasing at $\sigma = 0.903$ or later restores the after-contact window to 0.98 or above, and it stays there through the latest release point tested, $\sigma = 0.555$. The same sweep on a ball dropped onto a floor never breaks at all — after-contact horizontal agreement is 1.00 at every release point including the earliest, and the vertical agreement, which is the informative direction for a straight-down drop, is lowest at the earliest release (0.78 at $\sigma = 0.952$) and rises to 0.99 or above once the release is at $\sigma = 0.903$ or later. A ball rolling off a ramp holds 0.99 or better at every release point in both directions, but only a third of its frames are measurable at all, so it constrains little. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies.

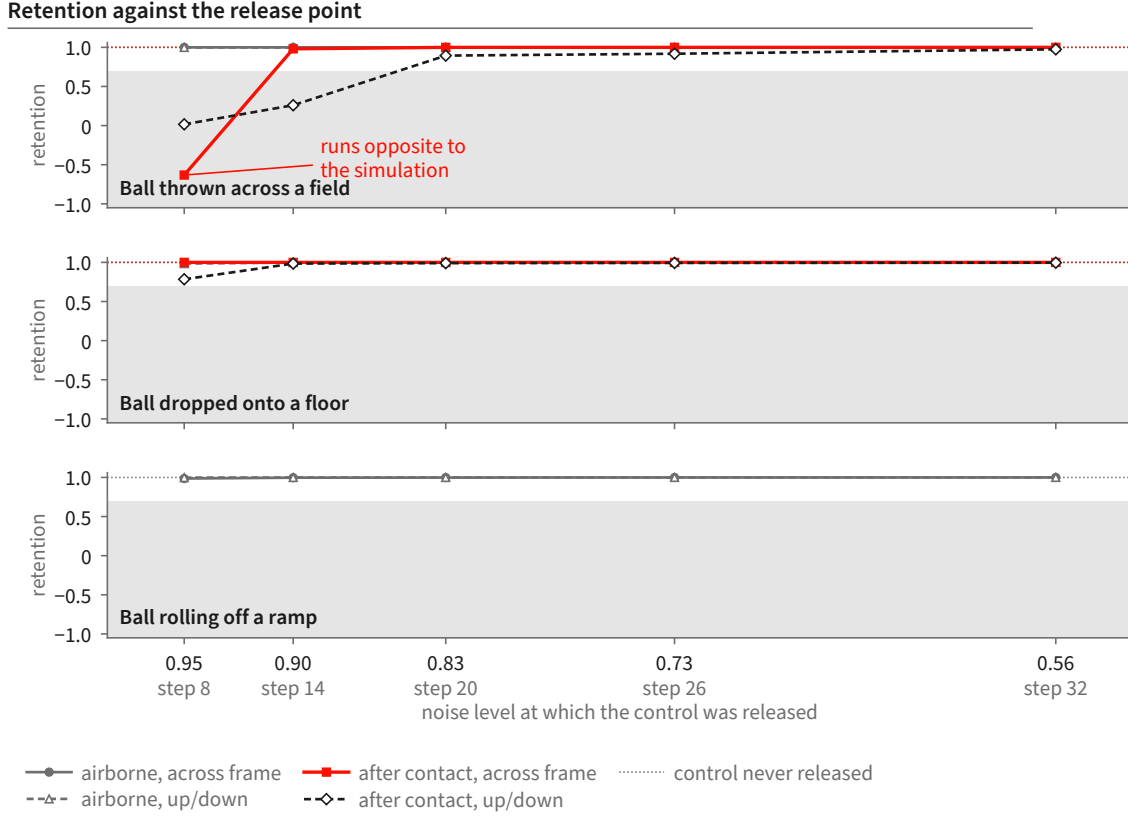


Figure 6: The release-point sweep: one clip per point, three scenes, scored separately over the airborne window and the window after the simulated object first reaches the ground. The horizontal axis is the noise level at which the control was switched off, high noise on the left; the step index of each run is printed underneath as a label only, because the same step index sits at a different noise level under a different schedule. The thrown ball’s after-contact horizontal motion is the one curve that breaks, and it breaks only at the earliest release: it runs *opposite* to the simulation at $\sigma = 0.952$ and is restored from $\sigma = 0.903$ onward. Its airborne flight is unaffected at every release point. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies. Grey band marks the region below the pre-registered 0.7 break line.

A denser sweep, and what it does not settle. A third run sweeps three release points on the thrown ball at three seeds and two guidance strengths, 18 clips, and scores them against the control with a segmentation-based tracker rather than a colour tracker. It reproduces the direction of the effect — airborne horizontal agreement is 1.00 (mean over the 6 clips at $\sigma = 0.952$, 95% CI 0.998 to 1.000), while after-contact horizontal agreement at that same release point averages 0.69 with a confidence interval spanning zero (-0.04 to 1.43 , $n = 6$) and is negative on one clip; at $\sigma = 0.903$ and $\sigma = 0.833$ the after-contact window recovers to 0.99 and 1.00. But it also shows the ceiling on this measurement: at the latest release point the tracker loses the object entirely on 3 of 6 clips, so those cells rest on 3 clips, not 6. On inspection those are clips in which the generated ball is genuinely tiny and low-contrast against the grass, not clips in which the tracker mis-fired, but either way the confidence intervals at the late release points are too wide to place the transition sharply.

Scope. One limit governs how far these numbers carry. The runs measured here condition the backbone by carrying a rendered source video through the denoising process and releasing it partway, rather than through the dedicated control branch that section 2 describes as the deployed route. The question—at what noise level does the low-frequency motion commit, and what breaks when the guidance is let go before then—is the same question in both cases, and the schedule is the same schedule; but these are measurements of the source-carrying route, and we do not claim the release point transfers unchanged to the control branch. Beyond that: these are single moving rigid bodies — a ball thrown across a field, a ball dropped onto a floor, a ball rolling off a ramp — at one resolution and frame count. The seeded part of the measurement covers one release point in depth (16 seeds), and the shape of the curve across release points rests on one seed per point for three scenes plus three seeds per point for one scene. Nothing here measures a multi-object collision, and nothing here measures the deformable and granular scenes of section 2.5, whose objects the tracker cannot follow as a single path. We also do not separate the release itself from the sampler-history reset that accompanies it in one of the two variants: the two variants are statistically indistinguishable on every window and direction we measured (the largest per-seed difference is 0.04 horizontally over the whole clip, 95% CI -0.04 to 0.13 , $n = 16$), which is consistent with the reset doing nothing once the motion has committed, but is not a demonstration that it never matters.

3 A Human-Annotated Video-Flaw Benchmark

3.1 Method summary

The benchmark starts from a corpus of AI-generated clips in which a person watched each clip against its prompt and wrote down every place the video failed to deliver what the text asked for, marking each failure with a severity, a confidence, the time range it occupies, a box track locating it in the frame, the exact prompt fragment it violates, and a sentence of reasoning. From that corpus we derive the evaluation target in three filtering steps, all recorded and re-runnable: keep the clips whose prompt or whose flagged failures describe motion and dynamics rather than, say, subtitle typography; keep the individual failures a video-only critic could in principle see, discarding the ones that are complaints about sound; and draw a smaller frozen core by stratified sampling under a fixed seed so that every experiment is scored against exactly the same target. The human-written fields are the ground truth. The coarse type tags used for reporting are assigned by models, not by the annotator, and are labelled as such wherever they appear.

3.2 The corpus

The benchmark is built from a corpus of **1,500 AI-generated video clips**, each carrying human annotations of where the video fails to match its prompt. The clips are AI-generated; the generating model is not recorded in the data, and we do not attribute them to a specific system. Every clip has at least one human-flagged flaw (table 2). This corpus is distinct from existing physical-commonsense benchmarks such as VideoPhy and its action-centric successor [5, 6], which we treat as separate transfer targets rather than as the annotation source here.

3.3 Human annotation schema

Each human flaw is a localized record with six fields (fig. 7). The annotator gives a **severity** and a **confidence**, each on a five-point scale; a **time span**, as a start and end frame index bounding the interval in which the flaw is visible; a **box track** locating it in the frame, recorded as labelled boxes at the beginning, middle and end of the flagged interval, in normalized image coordinates, with one box per object involved; the exact **prompt fragment** the video violates, quoted verbatim from the prompt; and a free-text **rationale** saying what went wrong. Each clip additionally carries an overall severity for the whole video and a short list of the objects, actions and relations the prompt calls for.

This is a richer object than a clip-level preference score: it says *what* is wrong, *where* in the frame, *when* in time, and *against which part of the prompt*. Not every field is filled on every flaw, and we state how often rather than assume: every flaw carries a quoted prompt fragment and a rationale, about seven in eight carry a frame range, and about three in four carry a box track—the remainder are explicitly marked as having no box, typically because the complaint is about the clip as a whole or about something absent rather than something visible in a particular place.

These human fields are the ground truth for everything in this report. Two coarse tags used for grouping and reporting—a *modality* tag saying whether the complaint is about what is seen, what is heard, or both, and a *type* tag naming the dominant kind of complaint from a fixed ten-item list—are *not* written by the annotator. They are assigned by asking three language models to classify each flaw from its quoted prompt fragment and rationale at temperature zero with a fixed seed, and taking the majority; a three-way disagreement on modality resolves to “both”, a three-way disagreement on type resolves to “other”, and a flaw that receives no usable vote falls back to a frozen keyword rule. We label these two fields as machine-assigned wherever they are used and never treat them as ground truth. Annotations are single-annotator: the corpus does not carry multiple independent annotations of the same flaw, so no inter-annotator-agreement figure is available and none is claimed.

3.4 The physics-focused evaluation set

The critic is evaluated not on all 1,500 clips but on a frozen, physics-focused subset. Selection runs in three recorded stages, and we state each because the denominator of every later measurement depends on them.

Stage one, clip selection. A clip enters the pool if its video file is actually retrievable and if it shows signs of dynamics. The dynamics test is a fixed vocabulary of motion words—falling, bouncing, rolling, colliding, pouring, toppling, and so on—applied to the prompt and, separately, to each flaw’s rationale and quoted prompt fragment. A clip is kept when its prompt contains at least one such word, or when one of its flaws does, or when it has a time-windowed flaw against an action-typed object. The test is a keyword rule, and we treat it as one: it selects for prompts and complaints that *talk about* motion, which is a proxy for, not a guarantee of, a flaw whose adjudication needs physical reasoning. This stage keeps **606** of the 1,500 clips, carrying **1,171** human flaws; most of the loss is clips the rule judges non-dynamic, with a smaller number dropped for having no retrievable video.

Stage two, flaw selection. Within those clips, a flaw is a usable detection target only if a critic that sees only pixels could in principle find it. We therefore keep the flaws whose modality tag is “seen” or “both” and drop the ones that are purely complaints about sound, which removes **64** flaws

What one human flaw record contains

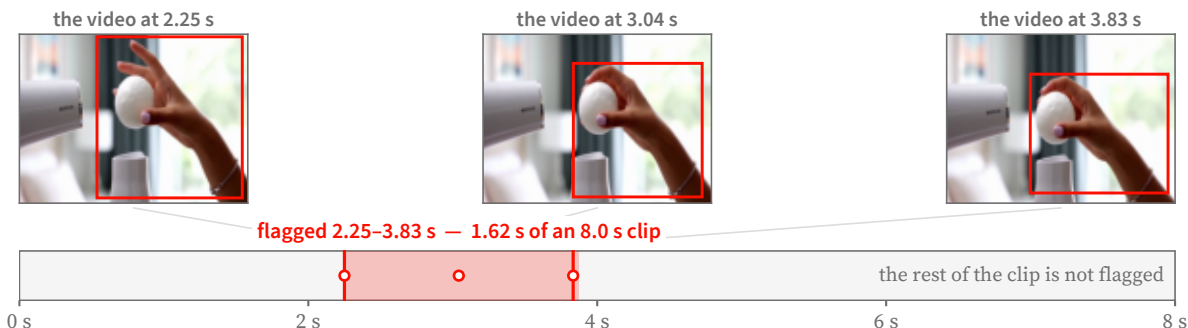
one of the 306 records in the evaluation core

THE PROMPT THE VIDEO WAS ASKED TO SATISFY

A ping pong ball balanced on a hair dryer's air stream, wobbling but never falling. A hand tries to grab it and misses. Audio: Dryer whirl, laughing.

the fragment the video fails to satisfy

WHERE THE FLAW OCCURS



HOW BAD IT IS, AND WHY

severity ●●●○○ 3 of 5 annotator confidence 5 of 5

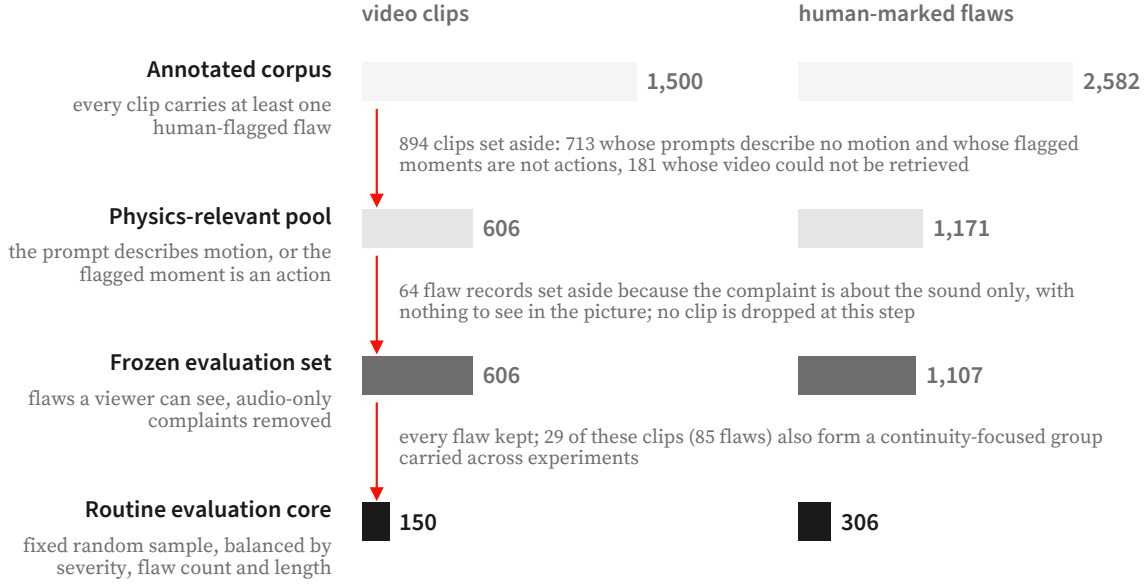
the annotator's own reasoning, as written

"The prompt requires 'a hand tries to grab it but fails', meaning the object should not be caught. However, it is caught in the actual video, which does not match the requested result."

Figure 7: One human flaw record, shown in full. The annotator quotes the prompt fragment the video fails to satisfy, bounds the interval in which the failure is visible, boxes the objects involved at the start, middle and end of that interval, and writes a severity, a confidence and a rationale in their own words. Frames and boxes are the annotator's; the flagged window is 1.62 s of an 8 s clip. Nothing about the critic's output appears here.

and leaves **1,107** as the full evaluation set. Three properties make the surviving records usable as targets rather than as opinions: the complaint is anchored to a verbatim prompt fragment, so what was promised is unambiguous; it is bounded in time, so a detector's claim can be checked against the interval a person marked; and it carries a severity, so misses can be reported by how much they mattered instead of counted flat.

Stage three, the frozen core. From the full evaluation set a smaller core of **150 clips with 306 flaws** is drawn for routine measurement. The draw is deterministic and its contract is recorded rather than left to a library default: a fixed seed; strata formed by crossing overall clip severity, number of flaws per clip, and clip duration; a largest-remainder proportional allocation across strata; within each stratum, candidates ordered lexicographically and then shuffled once by a single generator whose call sequence is fixed; and a replacement rule, drawing from the same stratum's queue first, for any clip that cannot be downloaded or decoded. A continuity-focused stratum of 29 clips and 85 flaws is carried inside the core so that measurements remain comparable to earlier work on the same clips, and it is exempt from replacement. Every clip that enters the core is verified end to end—downloaded and decoded—before the core is written, and the selection, the strata, and any replacements are stored with the set. Table 2 summarizes the counts, and fig. 8 shows the same derivation as a sequence of steps. We are explicit that the evaluation set is the 606-clip / 1,107-



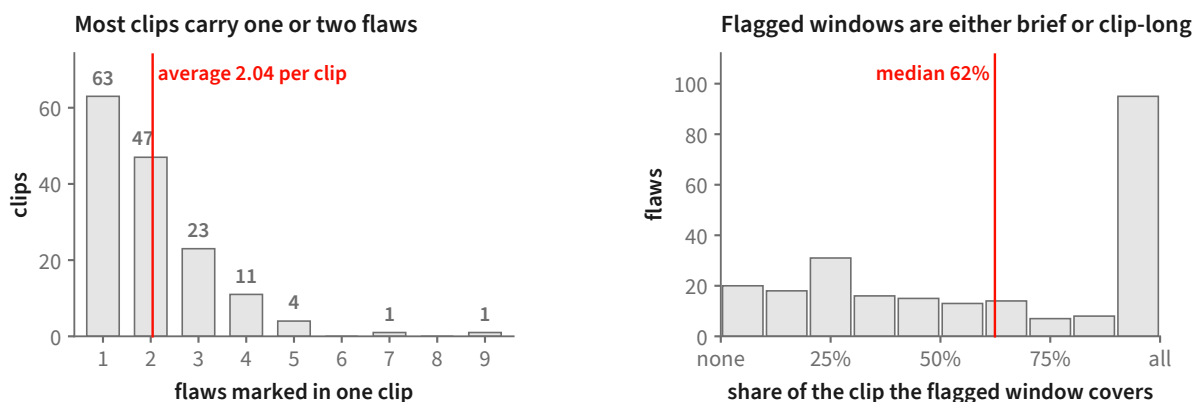
A further 711 annotated clips (1,130 flaws) are held back for training and share no clip with the frozen evaluation set or its routine core. Bar lengths are to scale within each column.

Figure 8: How the evaluation set is derived from the annotated corpus. Clips are removed only at the first step; the second step removes flaw *records* whose complaint is audible but not visible, leaving the clip count unchanged. The routine core is a fixed random sample balanced by severity, flaw count and clip length. A further 711 annotated clips are kept aside for training and share no clip with either frozen set.

Set	Clips	Human flaws
Full corpus (all AI-generated, all annotated)	1,500	2,582
Physics-relevant pool	606	1,171
Full evaluation set (frozen)	606	1,107
Routine evaluation core (frozen)	150	306
continuity stratum within the core	29	85
Disjoint training pool (excludes the frozen sets)	711	—

Table 2: Benchmark statistics, from the dataset manifests. The full evaluation set drops a small number of audio-scoped flaws relative to the physics-relevant pool. The training pool excludes every clip in the frozen evaluation sets.

flaw derivative (with a frozen 150-clip / 306-flaw core), not the full 1,500-clip corpus; conflating the two would overstate the evaluation. We are also explicit that the 150-clip / 306-flaw core is used as an *optimization set*: system and prompt decisions are repeatedly made against it, so coverage on it measures fit to that core rather than generalization, and a generalization claim requires a separately sequestered held-out set that has not influenced system design, which is future work.



150 clips of the routine evaluation core, 306 human-marked flaws. The right-hand panel uses the 237 flaws that carry a start and an end frame; the remaining 69 were marked without one. Of those 237, 57 fit inside a quarter of their clip and 95 run almost its whole length; the median flagged window lasts 5.00 s.

Figure 9: Two properties of the routine evaluation core that shape how a detector must behave. **Left:** most clips are flagged in only one or two places, so a detector that raises many complaints per clip will mostly be raising ones no annotator marked. **Right:** flagged windows are bimodal — many failures are confined to a small part of the clip while a comparable group runs almost its whole length — so both a brief localized check and a whole-clip check are needed.

3.5 Recall-oriented evaluation and a held-out training pool

Two properties of the core shape how a detector must behave (fig. 9). Because every clip in the pool carries at least one flaw and there are no clean clips, the benchmark measures *recall*—did the critic find the human flaws—and cannot on its own measure precision or false-alarm rate; a “flag-everything” detector would score perfectly on recall. We state this limitation plainly and address false alarms in the protocol of section 6. For any learning that consumes this data, a training pool of 711 clips is held strictly disjoint from the frozen evaluation sets, so the evaluation core is never trained on.

4 The Auditable Multi-Tool Critic

4.1 Method summary

The critic takes a prompt and a video and returns one of three answers—plausible, implausible, or abstain—together with the trace that produced it: the claims it checked, what each tool measured, and the rule that combined those measurements. The method has four stages (fig. 10). A served vision-language model reads the prompt only, splits it into small separately checkable claims, and writes down, for each claim, which measurements would settle it. Those requested measurements are type-checked as a program before anything runs. Each surviving measurement is then executed by a frozen open perception specialist that is contractually allowed to report only what it measured—a box, a count, a track, a time window, a depth—and never whether the prompt was satisfied. Finally a deterministic layer turns the measurements into per-claim outcomes and the per-claim outcomes into the clip answer, by fixed rules with no learned component. A sin-

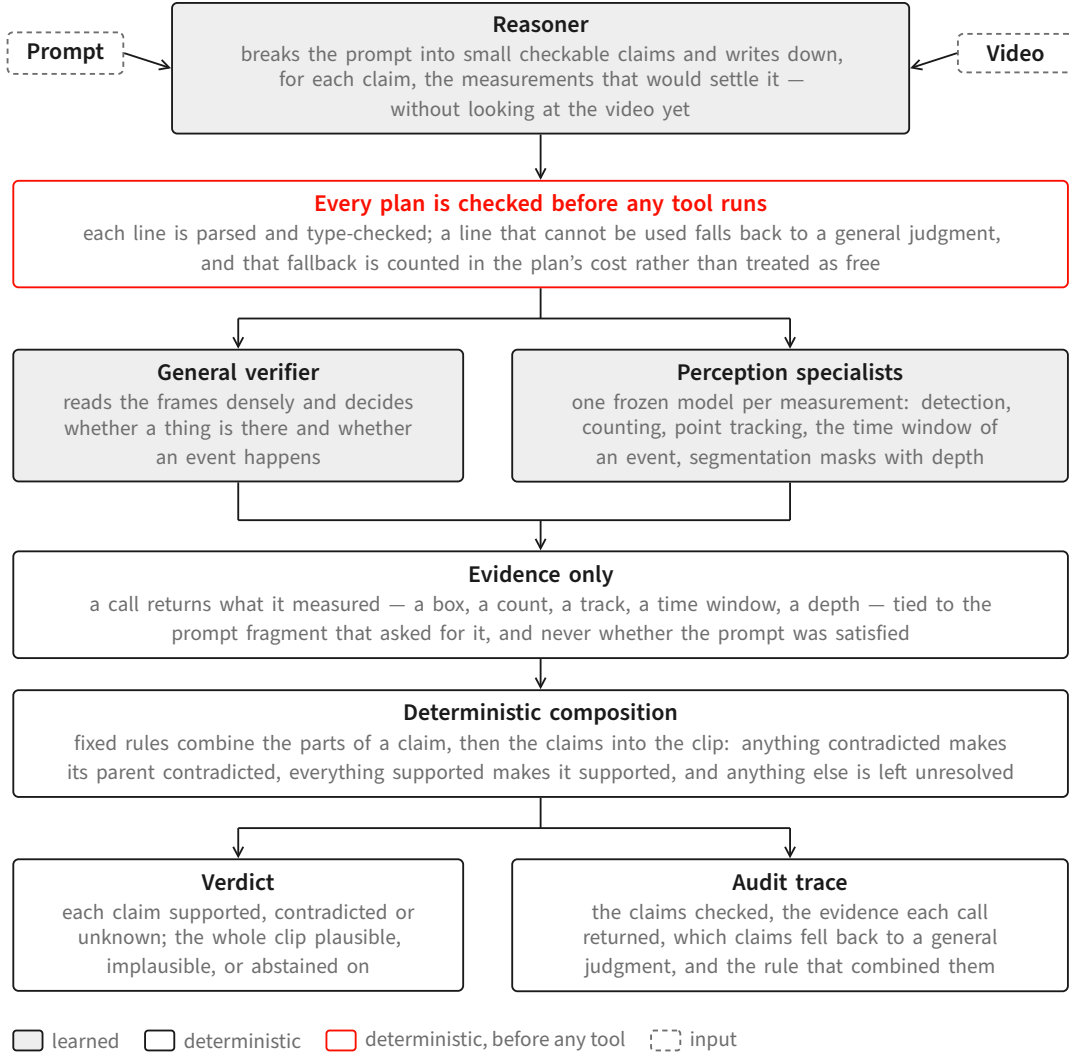


Figure 10: The critic. A reasoner turns the prompt into checkable claims and writes down the measurements that would settle each one; the plan is type-checked before any tool runs; a general verifier reading the frames densely decides whether things are present and events happen, while frozen specialists supply the quantities it cannot produce. Every call returns evidence only, and fixed rules — not a second model — turn that evidence into the verdict and the trace beside it. Shaded boxes are learned components; outlined boxes are deterministic.

gle holistic “is this plausible?” score is deliberately absent from that path, because it would hide which observation drove the answer and would let photorealism stand in for prompt satisfaction. The learned parts are the reasoner and the specialists; everything that combines evidence is deterministic and inspectable.

4.2 The reasoner

One served vision-language model—Qwen3-VL-30B-A3B-Instruct [39], BF16 weights (no quantization), tensor-parallel over two GPUs—performs the decomposition and grounding and runs three reasoner-mediated hard sub-checks (section 4.6). It is the only learned component that reasons

Capability	Open backend	Evidence returned
Object presence	LLMDet open-vocabulary detector [13]	presence; bounding box
Object count	CountGD-Box counter [2, 3] (SAM 2.1 internal)	cardinality of a named set
Kinematics	point tracking, TAPNext-style [59] + kinematic predicates	per-frame positions; static/-moving, direction, speed
Action timing	the reasoner over timestamped frames (a dedicated temporal grounder [58] remains available on request)	time window of a named event, or a stated refusal
Spatial relation	unified SAM 3 masks [8] + monocular depth + reasoning	depth-ordered / image-plane relations
Holistic	<i>no tool by design</i> [20]	—
Audio [†]	FlexSED sound-event detection [18]	sound events with timestamps

Table 3: Perception specialists and the open backend wired to each. In the routed configuration the object, count, kinematics, action, and spatial-relation backends execute as stages; the object and count adapters declare stronger primaries that are not on the executed path, so the running backend is shown. The relation capability is served by a unified mask-plus-depth reasoning arm, and no holistic scoring tool is used in the verdict path by design. [†]The audio backend is wired in the typed-plan path and is not yet exercised in the routed configuration.

over the whole prompt.

4.3 Perception specialists

Each claim is routed to a frozen open specialist that measures one thing and returns only that measurement (table 3). Two honesty notes are built into the table. First, some adapters *declare* a stronger primary backend that is not the one actually executed; we list the backend that runs. Second, there is no holistic scoring tool in the verdict path by design: a learned holistic video scorer is kept out so that the verdict is built from localized evidence and deterministic rules, though such a scorer could still serve as an external baseline.

4.4 Evidence-only tool contract

Specialists are frozen, and what they are permitted to say is fixed by a schema rather than by convention. A specialist reports *what it measured*—a box, a count, a track, a time window, a depth—and never *whether the prompt is satisfied*. The schema is enforced by a validator that refuses, rather than repairs, anything off-contract, so an off-contract output cannot enter the accepted evidence record at all. The complete rule set is:

- **Judgment-shaped fields are refused by name.** A fixed list of field names is rejected anywhere in a measurement, at any nesting depth—among them *label*, *judgment*, *decision*, *plausible*, *reasoning*, *rationale*, *ground truth*, *human label*, and *flaw*—as is any field name containing *mismatch* or *verdict*, and any name that starts like an evaluation label.

- **Renaming does not evade the rule.** Field names are normalized before the check: case and separators are stripped and camel-case is split, so a rejected name cannot be smuggled in by rewriting it.
- **Numbers must be real numbers.** Measurements are validated as strict data: no infinities, no not-a-number, no duplicate keys, no values outside the serialisable types. A measurement that cannot be canonically serialised is refused.
- **Every measurement is tied to the text that asked for it.** A record carries the clip identifier, the full prompt, and the exact claim fragment being checked, and that fragment is required to occur *verbatim* in the prompt—a paraphrase is a validation failure. This is what makes a measurement attributable after the fact.
- **Not-measuring is a first-class, and separate, outcome.** A record’s status is exactly one of measured, abstained, or errored. A measured record must carry measurements and may carry neither an abstention reason nor an error message; an abstention must carry a reason and no error; an error must carry a message and no abstention reason. The three cannot be blurred.
- **Heavy evidence is content-addressed.** Any file a specialist produces is referenced by path plus a hash of its bytes, and the measurement payload itself carries a hash of the tool’s native output, so a record can be checked later against the artifact it claims to describe.
- **Cost travels with evidence.** Every record carries its own accounting: accelerator seconds and tool calls are required; wall time, price, and token counts are optional. Cost is therefore auditable per measurement rather than reconstructed afterwards.
- **The contract is versioned.** Each record names the schema version it was written against, and a reader refuses a version it does not implement.

The guarantee this buys is narrow and worth stating precisely: no specialist *emits* the verdict, and every measurement that reaches the deterministic layer is attributable to a prompt fragment and to a hashed artifact. It does not make a learned measurement semantically neutral, and it cannot detect a measurement that is confidently wrong; a wrong number inside a well-formed record will mislead the rule that consumes it.

4.5 Three-valued semantics and deterministic roll-up

Every claim resolves to supported, contradicted, or unknown, and unknown is first-class: it is what the critic reports when the evidence it actually obtained does not settle the question. The rules that produce those three values are fixed and are stated here in full, because which of them applies is the whole substance of the critic’s answer.

A measurement that looked and did not find is evidence, not silence. When a specialist is asked about something over the whole clip and reports that it is not there, that is treated as a measurement and it contradicts the claim; it does not become unknown. Concretely: a stated count that does not match the measured count contradicts; a spatial claim whose two participants are never both visible with usable outlines contradicts, because the asserted arrangement was never realised anywhere; an asserted approach or departure whose subject cannot be followed at all contradicts; and a claim that something enters a container is contradicted both when it is never inside and when it is inside from the first observed frame with no crossing ever seen.

Existence and occurrence are decided by the dense-frame model, not by the specialists. A detector that returns a box, or a temporal grounder that returns a window, is not by itself evidence that the queried thing exists: those interfaces return their best candidate for a query whether or not the query has a referent. So whether an object is present at all, and whether a named event happens at all, are routed to the vision-language model watching the frames densely, and the specialists supply the quantities, windows, outlines, depths and tracks it cannot produce. A detector’s box remains an *input* to measurements—seeding a track, naming the participants of a spatial relation, feeding a count—and in that role its looked-and-absent report still contradicts. A measurement whose only consumers were existence or occurrence questions is not executed at all, which keeps unused perception out of the cost.

This is not a precaution against a hypothetical failure. Asked to time eleven events that a human viewer had confirmed do *not* occur in their clips, a dedicated temporal grounding model returned a confident time window for all eleven—placing tigers crouching across seven seconds of an eight-second clip, and a candle going out in a clip where the flame burns to the end—while the same reasoner that watches the frames, asked the same eleven questions in a form that permits refusal, said the event was absent in seven of them. On a control set of six events that do occur in the same clips, the reasoner found five and the grounder six, so the difference is not a reasoner that refuses everything. Where a window *was* readable the grounder ordered events perfectly; its failures were unusable output presented as measurement. Consequently the event-timing question is now put to the reasoner over frames labelled with their timestamps, in a form whose permitted answers include *absent* and *present but not placeable in time*; the dedicated grounder remains available when a caller asks for it. Two windows that begin at the same instant, or that each span the whole clip, are marked as carrying no order and the ordered claim is left unresolved rather than decided by a tie.

The comparison is small—eleven absent events and six present ones, on clips a single viewer had annotated—so it fixes the routing of one question rather than ranking two models in general, and it says nothing about how often either backend is right when an event is present and placeable. What it does establish is the asymmetry the design turns on: a backend that cannot decline will answer an unanswerable question, and an invented window is indistinguishable from a measured one downstream.

Permitting refusal is not free, and one traced case illustrates the cost. A claim asserted that a sound coincided with each impact. Because the input carried frames but no audio, the reasoner declined to give the sound a time window, and the ordering was left unresolved. The dedicated grounder returned a window for the same query despite having no audio evidence, and that unsupported estimate led to a flag that matched a human-labelled mismatch—so replacing it with a refusal lost one catch in this case. We prefer the trade, because unsupported measurements weaken the provenance of the detections a system takes credit for; but the trace establishes a concrete cost, not its aggregate size. Across twenty-six event-timing calls in that run the reasoner classified the queried event as absent in fourteen and declined to place it in one—the first being a judgement, the second a refusal, and only the second capable of costing a catch this way.

Ordering is composed, never assumed. For a claim that one event precedes or falls inside another, both events must first be confirmed to occur by a backend able to observe them—the dense-frame model for anything visible, and nothing on this path for an event that is only audible, which is why such an event yields a refusal rather than a window. If either event is judged absent, the ordered claim is contradicted outright; if either cannot be placed in time, the claim is left unresolved.

Only once both are confirmed do the specialist's time windows decide the order, and they do so with a tolerance rather than a fixed threshold: the boundary slack is a quarter of the shorter of the two events, so "before" requires a gap that is resolvable relative to the events being ordered. Containment is directional—one interval inside the other, not merely overlapping it—and an interval that contains the other contradicts rather than abstains.

Where unknown genuinely comes from. It is reserved, and it is reserved for cases that are about the evidence rather than about the video. Two valid window measurements that cannot be reconciled within the tolerance yield unknown: that is the deterministic outcome of the ordering table, not a tool's opinion. So does an incomplete plan—a depth-ordering relation asked for without a depth measurement is reported as a gap rather than silently resolved from outlines alone. So does a required measurement that never arrived. So does a check that the planner emitted but bound to no claim text and no describable request, which is surfaced by name rather than absorbed. And so does an infrastructure failure, which is retried at the clip level and never scored.

Sub-checks compose by a fixed rule, and an unresolved part does not disappear. A claim may carry several sub-checks. The claim is contradicted if *any* of them is contradicted; it is supported only if *every* one is supported; in every other case—which is exactly the case where at least one part is unresolved and none is contradicted—the claim itself is left unresolved. An unresolved part therefore blocks a claim from being asserted as satisfied, but does not by itself accuse.

Negation is handled at composition, not inside measurement. A negated sub-check is written only around a whole assertion—"this does not happen", "this object is not present"—and never around a measurement. At composition time its outcome is flipped first, supported becoming contradicted and contradicted becoming supported, and the ordinary rule above then applies. The consequence is that the measurement rules never have to reason about a negated query, which is why negation around a count, an interval or a spatial relation is rejected at planning time rather than approximated.

The clip answer. The roll-up is a separate deterministic step over the per-claim outcomes: if any claim is contradicted, the clip is *implausible*; otherwise, if every claim is supported, it is *plausible*; otherwise the critic abstains. "Plausible" therefore means only that no checked claim was contradicted—a summary of the plan that was executed, not a certificate of complete physical verification.

Where an abstain-by-default rule does apply. The three reasoner-mediated sub-checks of section 4.6 are the exception, and they are asymmetric in the opposite direction from the measurement rules. Each may only report a mismatch on the strength of a positively contradicting feature it can actually see; if the features that would distinguish the two candidate readings are not clearly resolvable in the evidence it was given, it abstains, and an unparseable or missing answer also abstains. Absence of confirmation is never treated as a contradiction there.

4.6 Reasoner-mediated hard sub-checks

Three checks resist full symbolization and remain reasoner-mediated on tool-localized inputs: object subtype identity (is the detected object the specific kind named), dense action-event recall (did a named event occur, densely over time), and depth-ordered spatial relations. Each runs a fresh reasoner pass over evidence a tool has already localized, under the same asymmetric gate. We name them explicitly rather than imply a pipeline that is more symbolic than it is.

4.7 Compiling claims into an executable measurement plan

Routing could be a flat claim-to-tool table. To make the orchestration inspectable, we instead compile each prompt into a *verification plan*: a small, checkable program that names the measurements the claims require. The plan is validated before any tool runs, equivalent measurements are executed once, and the resulting evidence is combined by fixed rules. This subsection describes how a plan is written, checked, routed, and shared. It concerns *plan construction only*: nothing here tests video verdicts, flaw coverage, or false accusations, and nothing here bears on whether plan-based execution outperforms independent per-claim verification (section 6).

4.7.1 A restricted language for measurement plans

Each claim is represented by **one line of code** in a small, typed language—the code-as-plan idea used in visual programming systems [17, 46], here narrowed to a verification vocabulary. For claim c_N the planner writes `check( $c_N$ , <expr>)`, where the expression is a single call or several joined by `and`. The reasoner does not see the video at this stage; it is writing down *what would have to be measured*, not answering. Representing the plan this way avoids cross-referenced identifiers spread across several generated tables and allows the plan to be checked mechanically before execution—planner-format fragility being a reported difficulty in tool-controller systems [43]. Table 4 lists the callable operations and the check each becomes.

4.7.2 How a plan is constructed

Whatever surface the planner writes, the plan the executor consumes is a small set of linked tables: the things the prompt talks about, the separate occurrences of the events it asserts, the measurements to be made, the typed checks over those measurements, and the claim each check answers. When the planner writes one line per claim, the deterministic translator assembles those tables as it goes, giving one identifier to each distinct mention it encounters and reusing it thereafter. When the tables are written directly, that consistency has to be arranged explicitly, and the construction proceeds in two passes over the prompt (fig. 11). The reason is reference, not length.

Name the things once, then decide what to measure. The first pass reads the whole prompt and does nothing but name. It lists every distinct object or actor the claims mention and every separate occurrence of an asserted event, fixes an identifier for each, and records which participants each occurrence involves. Mentions the prompt itself distinguishes—a first and a second animal of the same kind—receive different identifiers, and an event asserted twice receives one entry per occurrence rather than one entry for the event type. The pass is required to return nothing else: no measurements and no checks. Only then does a second pass decide, claim by claim, what

Operation	Meaning	Becomes
<code>judge("assertion")</code>	a dense-frame pass over the clip decides the assertion	event occurrence
<code>exists("object")</code>	the object is visible at some point	existence
<code>eq(count("phrase"), N)</code>	exactly N are visible	count
<code>before(window(A), window(B))</code>	A ends before B	temporal order
<code>during(window(A), window(B))</code>	A falls within B	temporal containment
<code>rel("A B", relation)</code>	spatial relation between two segmented objects	spatial
<code>traj("phrase", relation, target)</code>	tracked motion of an object	trajectory
<code>sound("description")</code>	the described sound is asserted	occurrence [†]

Table 4: The planner’s measurement vocabulary. `count` and `window` are structural and legal only inside `eq`, `before`, or `during`. Relations are drawn from fixed vocabularies (for example *behind*, *in front of*, *above*, *contained in* for space; *moves toward*, *rises then falls*, *accelerates* for motion), and the three directed motion relations require an explicit target. Depth-ordering relations (*behind*, *in front of*) cannot be settled from masks alone, so they additionally require a depth measurement; a plan that omits it is rejected as incomplete rather than silently resolved. [†]The plan-based path has no dedicated sound-event check: `sound` is routed to the general verifier, so on this path it does not constitute an audio measurement. The audio specialists of table 3 are used in the routed configuration (section 4.7.8).

would have to be measured, and it does so against that finished list, which it may refer to but not extend.

The split exists to keep references stable. If each part of a long prompt chose its own names while also choosing its own measurements, two parts would name the same object differently, or give one name to two different objects, and a check written in one part could not be composed with a check written in another—an ordering check, for instance, is only meaningful if both events it orders are the events the rest of the plan is talking about. Fixing the names first makes them shared vocabulary: the second pass points at a name and cannot quietly mint a competing one for something already named.

Claims are handled in small groups. The claims are then taken a few at a time—four by default—in the order they appear in the prompt. A group is shown the whole prompt and the whole list of names, but only its own claims, and returns only the measurements, the checks and the claim-to-check bindings for those claims. Because groups are written independently, their internal identifiers would collide once the groups are put together, so the identifiers are made group-local mechanically rather than by request: every measurement and check a group produced is renumbered under that group’s own label and the references inside the group are rewritten to match. A collision is therefore impossible even if a group ignores the instruction to label its own output. Two things are deliberately not the planner’s to choose: the claim identifiers and the claim wording are the ones the claim list already fixed, and a group that leaves one of its claims without a check yields a complaint naming that claim rather than a plan quietly missing it.

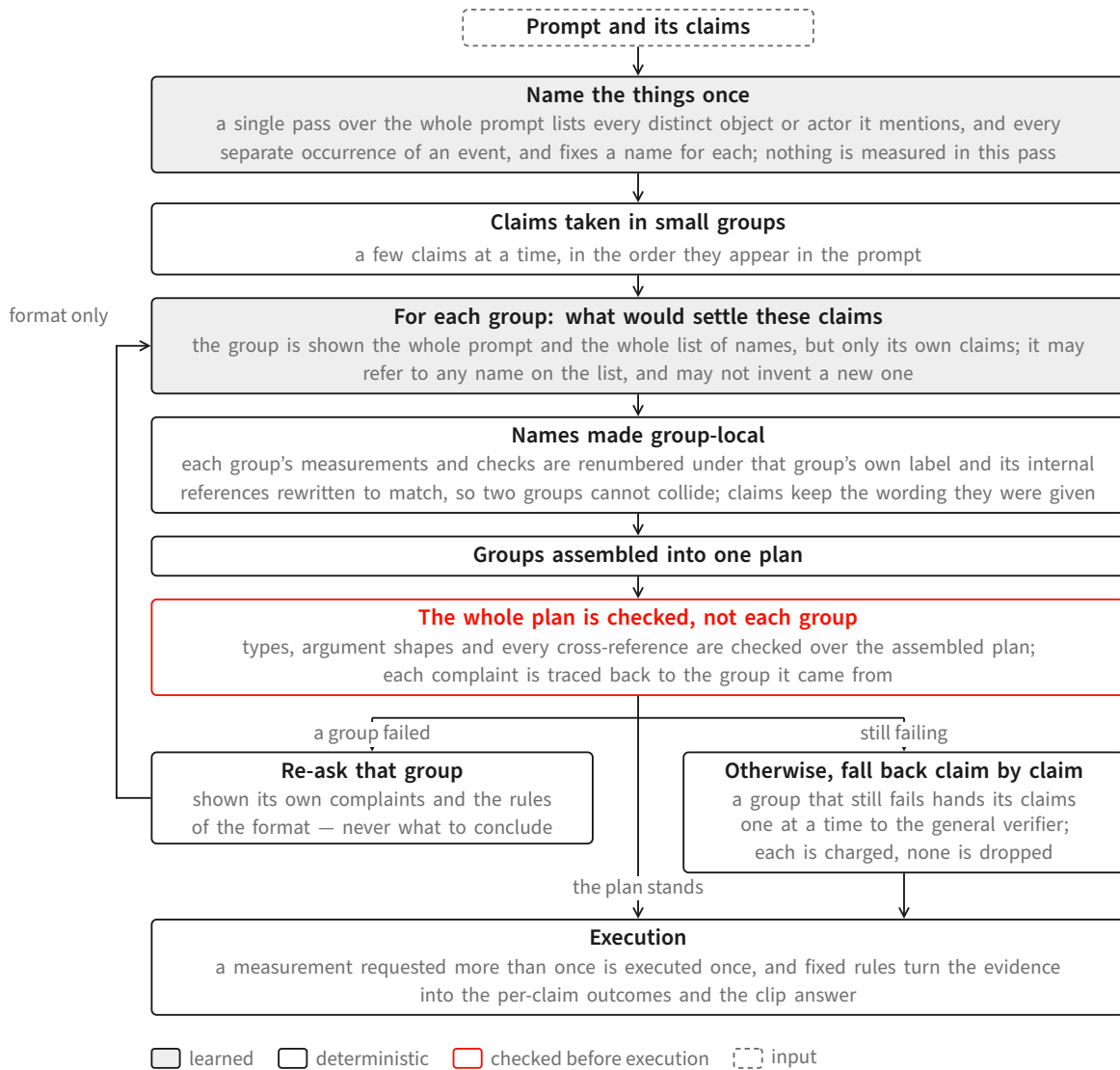


Figure 11: How a plan is constructed. One pass names everything the prompt talks about—the objects and actors, and each separate occurrence of an asserted event—and nothing else; the claims are then handled a few at a time against that fixed list, so that different parts of a long prompt refer to the same things. Each group’s identifiers are renumbered under its own label so groups cannot collide when assembled, and the assembled plan is checked as a whole, because what needs checking is mostly cross-reference. A group that fails is asked again with its own complaints and a restatement of the format, never with the answer; a group that still fails hands its claims to the general verifier one at a time, charged to the plan’s cost. Shaded boxes are learned components; outlined boxes are deterministic.

The assembled plan is checked as a whole, not group by group. Validation runs once, on the assembled plan. This is not a matter of convenience. Most of what needs checking is a reference from one part of the plan to another: whether a check points at a measurement that exists, whether a claim points at a check that exists, whether an event occurrence is bound to a participant that was actually named. A group inspected on its own cannot be checked for any of it, so group-by-group checking would accept every group and still admit an incoherent plan. Alongside

those references the whole-plan check enforces the same typing described in section 4.7: each kind of check must be fed measurements of the kinds that can settle it, relations must come from the fixed vocabularies, a stated quantity must be present, and a directed-motion check must carry a target. Each complaint the check produces is then traced back to the group whose identifiers it mentions.

Recovery is bounded, and it never supplies the answer. A group whose output drew a complaint is asked again, and is shown three things: its own previous output, the complaints against it, and a restatement of the format—the permitted measurement operations, the permitted kinds of check, the labelling convention for identifiers, and the requirement to cover every claim it was given. It is not told which measurement the claim should get, which relation is the right one, or what to conclude. That line matters: the choice of what to measure is the part of the system under study, so correcting it automatically would replace the behaviour being measured with the corrector’s behaviour and would hide the planning failures that should be counted. Teaching the format is not the same as supplying the decision, and only the former is done.

Recovery is bounded in both directions. The naming pass and each claim group are allowed a small fixed number of attempts. A group still failing after them is not dropped and is not left unresolved: its claims are handed to the general verifier one at a time, and each of those verifications is charged to the plan’s cost exactly like any other call. If the naming pass itself never produces a usable list, there is no shared vocabulary to write measurements against, and every claim of that prompt takes the same per-claim route. After any such degradation the remaining plan is rebuilt and re-checked, and if it still fails, its claims degrade the same way rather than an unchecked plan being passed on to execution.

What this buys and what it costs. What it buys is structural: names are agreed before anything is measured, so checks written for different parts of a long prompt refer to the same objects and events and can be composed; a failure is localised to a group and is either repaired or charged rather than silently shrinking the plan; and no plan reaches execution without passing a check over its complete set of cross-references. What it costs is also concrete. Construction takes several model calls rather than one—one for naming, one per group of claims, plus the bounded re-asks—and the naming pass is a single point of dependence for the whole prompt. A group sees only its own claims, so anything that has to cross between groups must travel through the shared list of names rather than through another claim’s wording. And every claim that degrades to per-claim verification is a claim whose plan was not used, which is why the accounting keeps those separate rather than folding them into the plan’s measurements. Whether this construction procedure helps a verdict, or costs less than any alternative, is not established here; the comparison that would settle it is the one specified in section 6.

4.7.3 Validating a plan before it runs

Because the plan is code, it can be checked mechanically before any tool is invoked. Each line is parsed; a line that does not parse, or that is not of the form `check(cN, ...)`, is rejected rather than guessed at. Only calls from the vocabulary in table 4 are permitted, and each is checked for arity and argument shape—a count must wrap a count term, a spatial relation must name two objects and a relation from the fixed vocabulary, a directed motion relation must carry a target. Logical

negation is accepted only directly around the three assertion-style operations, because a negated measurement would require uncertainty handling the downstream rules do not provide.

Failures are handled at two granularities, and neither is silent. If an entire line is unusable, the claim falls back to direct verification by the general verifier, and that fallback is *charged*—counted in the plan’s cost like any other call, so that fallback never looks free. If a single term is unusable—an invented relation, a directed motion without a target—only that term degrades to a generalist judgment; the substitution is recorded and explicitly counted as *not* a measurement, so that a plan cannot inflate its measurement rate by mislabeling salvaged terms. Every plan carries this accounting with it: how many calls, which claims fell back, which terms were salvaged.

4.7.4 Several checks per claim, and negation

A claim rarely reduces to a single test. Because an expression may join several calls, each conjunct becomes its own sub-claim with its own measurement, and the claim’s outcome is composed from theirs by a fixed rule: **if any part is contradicted the claim is contradicted; if every part is supported the claim is supported; otherwise the claim is left unresolved**. Negation is applied at this composition step rather than pushed into the measurement layer, so the measurement rules never have to reason about negated queries.

Composition is what lets one sentence be checked in more than one way (fig. 12). A claim that a ball triggers a cascade of a thousand mousetraps yields both a judgment that the cascade occurs *and* a count check against the number. Under the composition rule, a measured count below the stated number is by itself sufficient to contradict the claim even when the occurrence part is supported—a claim carrying only an occurrence check cannot express that.

4.7.5 Routing by what the claim says

Which measurement a claim receives is decided by its content rather than by a blanket rule. A numeral introduces a count check alongside the occurrence judgment; spatial prepositions introduce a spatial relation; directed or shaped motion introduces a trajectory check; ordering words introduce a temporal relation; a described sound introduces the audio operation. Anything else—and anything the planner is unsure about—is sent to the general verifier. That fallback guarantees the claim remains executable; it does not guarantee a correct verdict. Relations outside the fixed vocabularies are never invented; they fall back to a general judgment.

Routing interacts with a property of the specialists that we treat as load-bearing: **a specialist’s output is not by itself evidence that the queried thing exists**. Some specialist interfaces return their best candidate even when the query has no referent in the clip—a returned time window or bounding box can therefore reflect the query rather than the content. Existence and occurrence decisions are consequently assigned to the general verifier, which sees the frames densely, while the specialists supply the quantities, windows, masks, depths, and tracks it cannot produce. This assignment avoids treating a forced specialist output as proof of occurrence; it does not make absence judgments infallible. Composite claims combine the two: an ordering claim is first confirmed as two occurrences by the verifier, and only then ordered using the specialist’s windows.

4.7.6 Executing equivalent measurements once

Sharing happens at two levels, and we keep them distinct. Within a plan, repeated references may share one internal description, so a prompt mentioning the same object in five claims does not

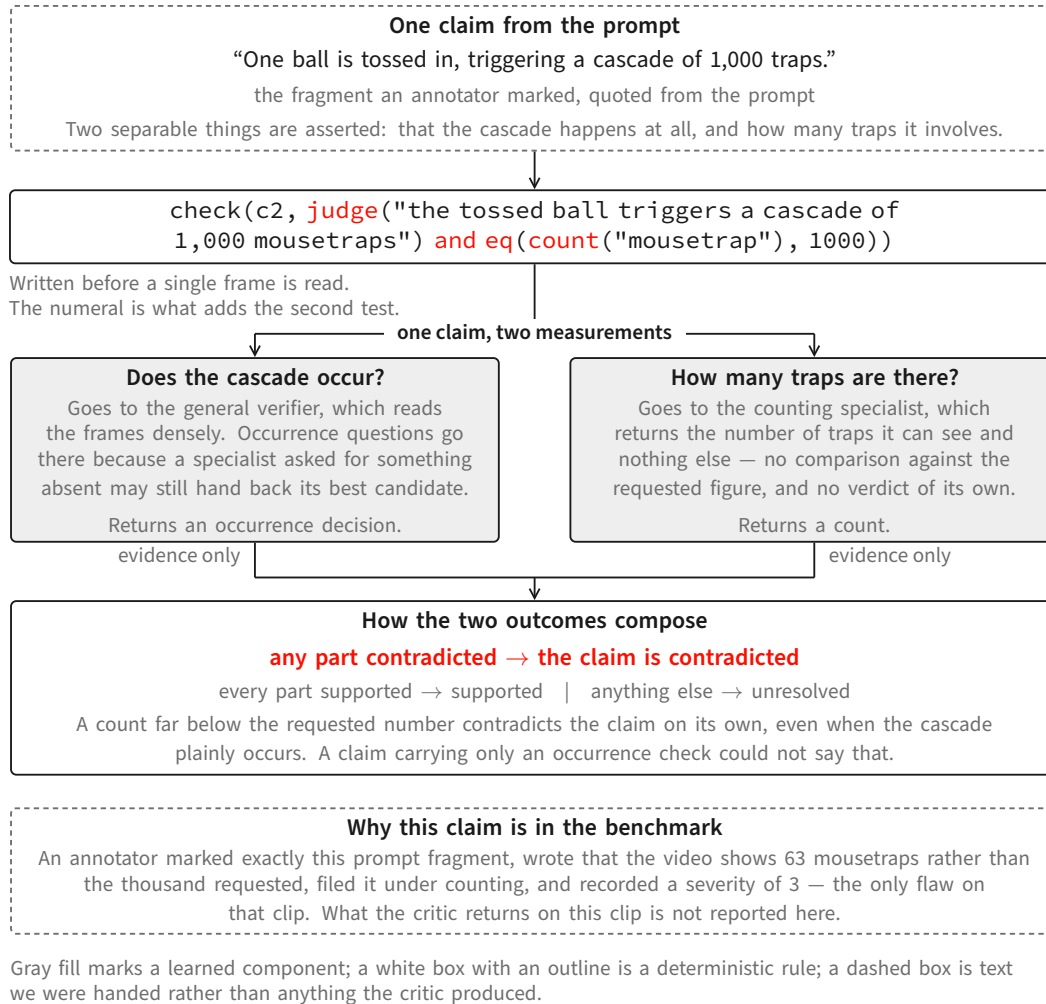


Figure 12: One claim, two measurements. A numeral in the claim adds a counting check beside the occurrence judgment, and the composition rule lets the count alone contradict the claim even when the cascade plainly happens. The prompt and the annotator’s record shown at the foot are real; what the critic returns on this clip is not reported here.

describe it five times. At execution, two measurements collapse into a single tool call only when the complete measurement request is identical—the operation, the referent, the exact query, the output kind, and the time range—so a whole-clip measurement and a sub-interval measurement are never merged. This is exact-request reuse, equivalent to common-subexpression elimination in the database sense [41, 42]; no sharing across merely similar queries is performed. Whether this yields a net saving in practice is a question for the evaluation, not a claim made here. Temporal relations remain directional and tolerance-scaled, following a subset of Allen’s interval algebra [1].

4.7.7 What the planner does not change

The planner is deliberately the only part that changes: a deterministic translator translates its lines into exactly the plan representation the compiler already consumes, so the compiler, the sharing rules, the measurement contracts, the execution layer, and the division between measuring and

judging are untouched. This separation is intended to keep the planner replaceable and to prevent a change of planning style from silently altering how evidence is interpreted.

4.7.8 Validation status and known gaps

The planner has been checked *at the planning stage only*: real model calls, no video, no tools. We define a claim as *measurement-carrying* when its validated plan contains at least one non-generalist operation that survives validation without being salvaged or falling back; a claim combining a general judgment with a count counts once, and salvaged terms never count.

On 233 claims drawn from 22 human-annotated clips, every claim produced a valid, type-correct, compilable plan—no fallbacks, no schema errors, no compile errors—at one to two model calls per clip. Measurement-carrying claims were 27/233 (11.6%), against 2.3% for the earlier tabular format on the same claims: a roughly fivefold relative increase that nonetheless leaves a large majority of these claims resting on a general judgment, because these prompts are dominated by occurrence-style assertions. On two separate diagnostic sets chosen to be measurement-dense, the rate was 19/30 (63.3%) on claims drawn from counting, spatial, and ordering flaws, and 10/10 on synthetic counting prompts. Those diagnostic sets probe the routing rules; they are not an estimate of coverage or verdict accuracy on the evaluation benchmark.

We are explicit about what this does *not* show. No video was processed, so it establishes that plans are valid, typed, compilable, and denser in measurements —*not* that any verdict improves. Effects on flaw coverage, on false accusations, and on cost are *unmeasured*; they are decided by the controlled end-to-end comparison of section 6, which contrasts plan-based execution with independent per-claim verification under matched cost accounting. A comparison between planners alone would not answer that question.

Known gaps: the spatial vocabulary has no “on top of” relation, so such claims degrade to a general judgment; the plan-based path has no dedicated sound-event check, so audio claims are not audio measurements on that path; negated measurements are unsupported by design; and outside the reported set, testing has exposed occasional out-of-vocabulary calls, which the validator converts into charged fallbacks rather than silently accepting.

Where specialists actually run. We distinguish two configurations. In the *routed* configuration each claim is dispatched to its dedicated specialist (table 3) and those tools run as executable stages. In the *plan-based* configuration the compiler and its typed checks run against a tool server that exposes the same specialists; in the runs completed so far, that path resolved most checks through the dense-frame generalist, both because occurrence-style claims route there by design and because the earlier planner rarely requested measurements—the specific limitation the planner described here is intended to address. Global plan selection among alternative tool recipes, and a value-of-information loop that spawns follow-up measurements, remain **PROPOSED**.

5 The Intended Refinement Loop

The three components are meant to close a loop: the benchmark trains and evaluates the critic; the critic scores generated clips; and the critic’s verdict becomes a reward that fine-tunes or guides the generator toward physical faithfulness. **This loop is not yet built.** The generator and the critic today run on disjoint data—the generator on simulation-driven clips, the critic on the human-annotated benchmark—and no reward-driven generation run exists. Prerequisites are a critic

whose reward is trustworthy on the frozen human labels (section 6) and a decision about where the reward enters generation (preference optimization, guidance, or data selection). We state the loop as the organizing goal and as explicit future work, not as a result.

6 Evaluation Protocol and Measurement

This section specifies how the critic will be measured; it deliberately reports no accuracy or coverage numbers.

Target and ground truth. The frozen physics-focused set of section 3 (150 clips / 306 flaws for routine measurement, 606 clips / 1,107 flaws for full sweeps). Human labels are the only ground truth. The 150-clip / 306-flaw core is the optimization set: because design decisions are repeatedly made against it, reported coverage measures fit to that core rather than generalization, and a generalization claim requires a separately sequestered held-out set that has not influenced system design.

Matching is outside the critic. To decide whether a critic output corresponds to a human-flagged flaw, a separate protocol uses two judge models (a GPT-family and a Gemini-family model, majority over runs). These judges perform *flaw matching only*; they are not components of the critic, do not produce its verdict, and are not treated as ground truth. Any “majority-of-three” in this project refers to such a model-vote protocol, never to human annotators.

Recall, and the false-alarm gap. Because the benchmark has no clean clips, the primary measurable quantity is recall of the human flaws. A meaningful false-alarm or precision number requires either clean clips or per-allegation adjudication of the critic’s non-matching outputs; we specify that adjudication as part of the protocol and do not substitute a recall number for it.

Dense-frame requirement. The critic must observe the video densely. The evaluated components extract frames densely under a logged sampling policy, and any measurement obtained from a sparsely sampled input is excluded. A re-run counts only if it uses the dense path on the unchanged frozen target with BF16 tools at pinned revisions.

The frozen confirmatory comparison. We evaluate the orchestration design on the 150-clip optimization set, which carries all 306 human-labeled flaws. The primary paired comparison estimates the joint effect of structured execution—shared measurements, routing to capabilities, composition across measurements, and conflict resolution—relative to a wording-controlled per-claim baseline on the same complete set. Because verification questions can be phrased in ways that produce different verifier behavior, the per-claim baseline uses fixed wording. An auxiliary control runs the per-claim baseline’s exact claim texts through the plan machinery; in the current build those claim texts are byte-identical, so this control measures added plan cost and run-to-run serving variability and cannot test a decomposition effect. A separate test of decomposition would require distinct claim texts and is deferred.

Because a detector call, a tracker call, and a full vision-language judgment have different costs, we record model and tool calls, processed frames, and GPU-seconds rather than counting every

invocation as equally costly. We report each system at its uncapped operating cost and make cost-matched comparisons in both directions: the structured system is capped at the per-claim baseline’s cost, and the per-claim baseline is extended to the structured system’s uncapped cost, using a priority policy fixed in advance. Allegations that do not match a human-labeled flaw count as *review burden*, not false positives, in the frozen-benchmark score; any separate adjudication of those allegations is reported only as an analysis of possible omissions in the benchmark labels, and does not expand the frozen labels or alter the frozen-benchmark score. We estimate uncertainty by resampling whole clips, because flaws within the same clip are dependent.

No comparative results are reported here. The final report will disclose that this set was inspected and used during optimization, that the wording-controlled per-claim baseline was introduced after treatment outputs had been observed, and that no retroactive held-out split exists within these 150 clips; a claim of generalization therefore requires evaluation on a new frozen set.

7 Related Work

7.1 Physics and structure in video generation

Physical-commonsense generation and its evaluation are studied by VideoPhy [5, 6], PhyGenBench [36], and Physics-IQ [38]; V-JEPA-style work argues physics is best judged in a learned state space [14]. Our generator does not learn physics: it *imports* it from a simulator and asks the backbone to render it, via structure conditioning. Structure- and motion-conditioned synthesis has a strong lineage—ControlNet for images [57], motion control for video [49], the Wan family and its VACE control interface [25, 48], and, as guided editing rather than control-branch conditioning, SDEdit [35]. DiffPhy conditions a diffusion backbone on a physics chain-of-thought [56]; we instead condition on a rendered depth field from a simulation.

7.2 Evaluating text-to-visual faithfulness

Decomposing a prompt into checkable pieces is established: TIFA [22] and the Davidsonian Scene Graph [9] score generated questions/propositions with a vision-language model, VQAScore uses an image-to-text model [31], and GenAI-Bench, VBench, T2V-CompBench, FETV, and EvalCrafter benchmark compositional alignment [23, 28, 32, 33, 45]; embedding and preference scorers predict alignment or human preference [21, 27, 53], and learned quality models predict human judgment directly [19, 20]. We keep such a holistic learned scorer out of the verdict path. Relative to these, our critic routes each claim to a dedicated open perception specialist under an evidence-only contract, validates a typed plan before execution, distinguishes event occurrences, reuses perception only under an exact contract, and preserves claim-level provenance with an explicit unknown outcome. We claim novelty for the integration, not the parts.

7.3 Visual programs, tool-use, and modular video QA

Composing perception modules for reasoning includes Neural Module Networks [4], VisProg [17], ViperGPT [46], CodeVQA [44], and neural-symbolic VQA [24, 55]; LLM tool-controllers and plan-execute graphs include HuggingGPT, TaskMatrix.AI, ControlLLM, LLMCompiler, and ReWOO [26, 30, 34, 43, 52], with search-based variants [7, 54, 60, 61]; modular video QA includes ProViQ, MoReVQA, the VideoAgent systems, and VideoTree [10, 12, 37, 50, 51]. Our execution layer is narrower and

verification-specific: a *typed* plan, type-checked before tools run, whose evidence obligations carry three-valued semantics; it does not search over alternative plans. Reuse of equivalent measurements is common-subexpression elimination in the multi-query-optimization sense [15, 16, 41, 42], and the specialists themselves draw on open detection, counting, tracking, grounding, and audio models [2, 3, 8, 11, 13, 18, 29, 40, 58, 59].

7.4 Flaw and preference datasets

Prior video benchmarks largely provide clip-level physical-commonsense or preference labels [5, 6, 36]. Our benchmark instead provides *localized* human flaw annotations—severity, time span, a box track, the violated prompt fragment, and a rationale—so that a detector can be scored not only on whether a clip is flawed but on whether it found the specific, spatially and temporally located error a person marked.

8 Limitations

Generation. Validation is qualitative (visual inspection plus a trajectory tracker); there is no learned quality metric for this line. Multi-object interaction is unsolved. The control signal comes from a simulator, so scene diversity is bounded by what is simulated, and the mapping from simulation to prompt appearance is not itself verified for physical consistency.

Benchmark. The corpus is recall-only (no clean clips), so precision is not directly measurable; flaws are single-annotator with no inter-annotator agreement; the scope/category tags are machine-derived; and the source generator of the clips is unrecorded. Repeatedly designing against one frozen set risks overfitting to it even without training on it.

Critic. The spatial relation predicate and negative-event verification are limited; the typed-plan layer does not yet dispatch the full specialist registry; binding remains a central failure mode, since the compiler enforces symbolic identifier consistency but cannot guarantee that heterogeneous tools localized the same physical instance; and the reasoner-mediated sub-checks reduce but do not remove semantic opacity.

Loop. The generator–critic loop is unbuilt, and critic quality on the frozen labels is the gate before any reward-driven generation is attempted.

9 Conclusion

Physical faithfulness in generated video requires steering, measurement, and ground truth together. This report contributes one piece for each: a generator that imports geometry and motion from a simulation and renders them through a video-diffusion control branch; a human-annotated benchmark that localizes real flaws in space, time, and prompt reference; and an auditable multi-tool critic that decomposes a prompt, routes each claim to an open specialist returning evidence only, and combines the evidence deterministically into a three-valued verdict. We have described each component and its honest implementation status and specified the human-label protocol under which the critic—and eventually the loop that lets it refine the generator—will be measured.

The comparative numbers, and the closed loop itself, are the next milestones, not claims of this report.

A Appendix

To be expanded: the full generation configuration and control-render details; the benchmark schema and per-stratum statistics; the critic’s plan schema, predicate type rules, and temporal-relation truth tables; the specialist license and revision manifest; and the pre-registered evaluation manifest.

References

- [1] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434.
- [2] Niki Amini-Naieni and Andrew Zisserman. Open-world object counting in videos. In *AAAI*, 2026. arXiv:2506.15368. The CountVid model; the count arm.
- [3] Niki Amini-Naieni, Tengda Han, and Andrew Zisserman. Countgd: Multi-modal open-world counting. In *NeurIPS*, 2024. arXiv:2407.04619.
- [4] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Neural module networks. In *CVPR*, 2016. arXiv:1511.02799.
- [5] Hritik Bansal, Zongyu Lin, Tianyi Xie, Zeshun Zong, Michal Yarom, Yonatan Bitton, Chenfanfu Jiang, Yizhou Sun, Kai-Wei Chang, and Aditya Grover. Videophy: Evaluating physical commonsense for video generation. *arXiv preprint arXiv:2406.03520*, 2024.
- [6] Hritik Bansal, Clark Peng, Yonatan Bitton, Roi Goldenberg, Aditya Grover, and Kai-Wei Chang. Videophy-2: A challenging action-centric physical commonsense evaluation in video generation. *arXiv preprint arXiv:2503.06800*, 2025.
- [7] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, et al. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI*, 2024. arXiv:2308.09687.
- [8] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Dídac Surís, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. Open-vocabulary concept segmentation/tracking; used by the relation depth arm.
- [9] Jaemin Cho, Yushi Hu, Roopal Garg, Peter Anderson, Ranjay Krishna, Jason Baldridge, Mohit Bansal, Jordi Pont-Tuset, and Su Wang. Davidsonian scene graph: Improving reliability in fine-grained evaluation for text-to-image generation. In *ICLR*, 2024. arXiv:2310.18235.
- [10] Rohan Choudhury, Koichiro Niinuma, Kris M. Kitani, and László A. Jeni. Zero-shot video question answering with procedural programs. *arXiv preprint arXiv:2312.00937*, 2023. The video analogue of ViperGPT.

- [11] Carl Doersch, Ankush Gupta, Larisa Markeeva, Adrià Recasens, Lucas Smaira, Yusuf Aytar, João Carreira, Andrew Zisserman, and Yi Yang. Tap-vid: A benchmark for tracking any point in a video. In *NeurIPS Datasets and Benchmarks*, 2022. arXiv:2211.03726.
- [12] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *ECCV*, 2024. arXiv:2403.11481.
- [13] Shenghao Fu, Qize Yang, Qijie Mo, Junkai Yan, Xihan Wei, Jingke Meng, Xiaohua Xie, and Wei-Shi Zheng. LlmDET: Learning strong open-vocabulary object detectors under the supervision of large language models. In *CVPR*, 2025. arXiv:2501.18954.
- [14] Quentin Garrido, Nicolas Ballas, et al. Intuitive physics understanding emerges from self-supervised pretraining on natural videos. *arXiv preprint arXiv:2502.11831*, 2025. V-JEPA intuitive physics: violation-of-expectation in a learned latent beats pixel/text-space.
- [15] Goetz Graefe. The cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.
- [16] Goetz Graefe and William J. McKenna. The volcano optimizer generator: Extensibility and efficient search. In *ICDE*, 1993.
- [17] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *CVPR*, 2023. arXiv:2211.11559.
- [18] Jiarui Hai, Helin Wang, Weizhe Guo, and Mounya Elhilali. Flexsed: Towards open-vocabulary sound event detection. *arXiv preprint arXiv:2509.18606*, 2025. Text-queryable sound-event detection with timestamps; the audio arm.
- [19] Xuan He, Dongfu Jiang, Ge Zhang, Max Ku, Achint Soni, et al. Videoscore: Building automatic metrics to simulate fine-grained human feedback for video generation. In *EMNLP*, 2024. arXiv:2406.15252.
- [20] Xuan He, Dongfu Jiang, Ping Nie, Minghao Liu, Zhengxuan Jiang, et al. Videoscore2: Think before you score in generative video evaluation. *arXiv preprint arXiv:2509.22799*, 2025.
- [21] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. CLIPScore: A reference-free evaluation metric for image captioning. In *EMNLP*, 2021. arXiv:2104.08718.
- [22] Yushi Hu, Benlin Liu, Jungo Kasai, Yizhong Wang, Mari Ostendorf, Ranjay Krishna, and Noah A. Smith. TIFA: Accurate and interpretable text-to-image faithfulness evaluation with question answering. In *ICCV*, 2023. arXiv:2303.11897.
- [23] Ziqi Huang, Yanan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *CVPR*, 2024. arXiv:2311.17982.
- [24] Drew A. Hudson and Christopher D. Manning. Learning by abstraction: The neural state machine. In *NeurIPS*, 2019. arXiv:1907.03950.

- [25] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. Vace: All-in-one video creation and editing. *arXiv preprint arXiv:2503.07598*, 2025. Mask-conditioned controllable video generation; the project’s stylizer.
- [26] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *ICML*, 2024. arXiv:2312.04511. Plans a DAG of function calls, dispatches independent calls in parallel.
- [27] Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. In *NeurIPS*, 2023. PickScore. arXiv:2305.01569.
- [28] Baiqi Li, Zhiqiu Lin, Deepak Pathak, Jiayao Li, Yixin Fei, Kewen Wu, Tiffany Ling, Xide Xia, Pengchuan Zhang, Graham Neubig, and Deva Ramanan. GenAI-Bench: Evaluating and improving compositional text-to-visual generation. In *CVPR Workshops*, 2024. arXiv:2406.13743.
- [29] Zeqian Li, Shangzhe Di, Zhonghua Zhai, Weilin Huang, Yanfeng Wang, and Weidi Xie. Universal video temporal grounding with generative multi-modal large language models. *arXiv preprint arXiv:2506.18883*, 2025. UniTime, the earlier action-arm temporal grounder.
- [30] Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, et al. Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis. *arXiv preprint arXiv:2303.16434*, 2023.
- [31] Zhiqiu Lin, Deepak Pathak, Baiqi Li, Jiayao Li, Xide Xia, Graham Neubig, Pengchuan Zhang, and Deva Ramanan. Evaluating text-to-visual generation with image-to-text generation. In *ECCV*, 2024. VQAScore. arXiv:2404.01291.
- [32] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tiejong Zeng, Raymond Chan, and Ying Shan. EvalCrafter: Benchmarking and evaluating large video generation models. In *CVPR*, 2024. arXiv:2310.11440.
- [33] Yuanxin Liu, Lei Li, Shuhuai Ren, Rundong Gao, Shicheng Li, Sishuo Chen, Xu Sun, and Lu Hou. FETV: A benchmark for fine-grained evaluation of open-domain text-to-video generation. In *NeurIPS*, 2023. arXiv:2311.01813.
- [34] Zhaoyang Liu, Zeqiang Lai, Zhangwei Gao, Erfei Cui, Ziheng Li, et al. Controlllm: Augment language models with tools by searching on graphs. *arXiv preprint arXiv:2310.17796*, 2023. Tool use as search over a tool graph (“Thoughts-on-Graph”).
- [35] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2022.
- [36] Fanqing Meng, Jiaqi Liao, Xinyu Tan, Wenqi Shao, Quanfeng Lu, Kaipeng Zhang, Yu Cheng, Dianqi Li, Yu Qiao, and Ping Luo. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. *arXiv preprint arXiv:2410.05363*, 2024. The PhyGen-Bench physical-commonsense generation benchmark.

- [37] Juhong Min, Shyamal Buch, Arsha Nagrani, Minsu Cho, and Cordelia Schmid. Morevqa: Exploring modular reasoning models for video question answering. In *CVPR*, 2024. arXiv:2404.06511.
- [38] Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? *arXiv preprint arXiv:2501.09038*, 2025. The Physics-IQ benchmark; visual realism does not imply physical understanding.
- [39] Qwen Team. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. The vision-language reasoner (Qwen3-VL-30B-A3B-Instruct) at the centre of the critic.
- [40] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- [41] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhole. Efficient and extensible algorithms for multi query optimization. In *SIGMOD*, 2000. Practical shared-subexpression MQO in a Volcano-style optimizer.
- [42] Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems (TODS)*, 13(1):23–52, 1988. Foundational MQO: optimize a batch of queries jointly via shared sub-expressions.
- [43] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. In *NeurIPS*, 2023. arXiv:2303.17580.
- [44] Sanjay Subramanian, Medhini Narasimhan, Kushal Khangaonkar, Kevin Yang, Arsha Nagrani, Cordelia Schmid, Andy Zeng, Trevor Darrell, and Dan Klein. Modular visual question answering via code generation. In *ACL*, 2023. arXiv:2306.05392.
- [45] Kaiyue Sun, Kaiyi Huang, Xian Liu, Yue Wu, Zihan Xu, Zhenguo Li, and Xihui Liu. T2V-CompBench: A comprehensive benchmark for compositional text-to-video generation. *arXiv preprint arXiv:2407.14505*, 2024.
- [46] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *ICCV*, 2023. arXiv:2303.08128.
- [47] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [48] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. The Wan 2.1 open video-diffusion backbone.
- [49] Xiang Wang, Hangjie Yuan, Shiwei Zhang, Dayou Chen, Jiuniu Wang, Yingya Zhang, Yujun Shen, Deli Zhao, and Jingren Zhou. VideoComposer: Compositional video synthesis with motion controllability. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [50] Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In *ECCV*, 2024. arXiv:2403.10517.

- [51] Ziyang Wang, Shoubin Yu, Elias Stengel-Eskin, Jaehong Yoon, Feng Cheng, Gedas Bertasius, and Mohit Bansal. Videotree: Adaptive tree-based video representation for llm reasoning on long videos. In *CVPR*, 2025. arXiv:2405.19209.
- [52] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023. Plan-up-front, then execute; the ReWOO plan-execute separation.
- [53] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. In *NeurIPS*, 2023. arXiv:2304.05977.
- [54] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023. arXiv:2305.10601.
- [55] Kexin Yi, Jiajun Wu, Chuang Gan, Antonio Torralba, Pushmeet Kohli, and Joshua B. Tenenbaum. Neural-symbolic VQA: Disentangling reasoning from vision and language understanding. In *NeurIPS*, 2018. arXiv:1810.02338.
- [56] Ke Zhang, Cihang Xiao, Yiqun Xu, Jiebo Mei, and Vishal M. Patel. Think before you diffuse: Infusing physical rules into video diffusion. *arXiv preprint arXiv:2505.21653*, 2025. The parent “DiffPhy” work this project extends. Title as verified live on the arXiv abstract page; some project docs cite an earlier subtitle “Physics-Aware Video Generation via Chain-of-Thought Conditioning.”
- [57] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [58] Yongdong Zhang, Yi Wang, Yixiao Ge, Yuying Ge, Xintao Li, Ying Shan, and Xiaohu Wang. Timelens: Rethinking video temporal grounding with multimodal llms. *arXiv preprint arXiv:2512.14698*, 2025. Closest published reference for the deployed action arm (the MCG-NJU TimeLens2-8B checkpoint has no standalone paper).
- [59] Artem Zhohus, Carl Doersch, Yi Yang, Skanda Koppula, Viorica Patraucean, Xu He, Ignacio Rocco, Mehdi S. M. Sajjadi, Sarath Chandar, and Ross Goroshin. Tapnext: Tracking any point (tap) as next token prediction. *arXiv preprint arXiv:2504.05579*, 2025. The point-tracking backbone behind the kinematic (physics) arm’s TAPNext++.
- [60] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*, 2024. arXiv:2310.04406.
- [61] Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models with a* search. In *ICLR*, 2024. arXiv:2310.13227.